Prabhāsa: Pāṇinian Structured Pretraining for Small Language Models

Sharath Sathish [1]

[1] *Department of Computer Science, University of York, York YO10 5GH, UK (alumnus)*

Correspondence: sharath.sathish@gmail.com

Abstract:
Data-efficient pretraining of small language models raises a sharp question: which inductive biases actually improve generalization under a fixed token budget, and which only appear to until they are tested with proper controls? We address this with Prabhāsa, a pre-registered, matched-budget study that asks whether Pāṇinian morphosyntactic structure, operationalized through Vidyut morpheme-boundary embeddings, kāraka role-stratified masking, and a supervised role-prediction objective, improves small-model pretraining on the official BabyLM suite. We train compact bidirectional encoders (roughly 114M parameters on the 10M-token Strict-Small budget and 100M parameters on the 100M-token Strict budget) and evaluate every claim with three or more seeds, paired bootstrap inference, and Holm–Bonferroni correction. Our central positive result is a scale-dependent effect of the pretraining objective: a pure masked-language-modelling (MLM) objective is statistically indistinguishable from a hybrid MLM + CLM objective at the 10M-token budget (63.58 versus 64.09 BLiMP, overlapping intervals) but outperforms it by 5.49 BLiMP points at the 100M-token budget (73.06 versus 67.57), nuancing the hybrid recipe of the BabyLM 2024 winner GPT-BERT. We report the 100M figure transparently as a single-seed point estimate: a multi-seed reproducibility analysis shows the recipe is not yet robust to the training seed at this scale (two further seeds, from identical initialization, settle at 53–58 BLiMP, with one optimizer trajectory diverging), so the magnitude of the 100M advantage is seed-sensitive rather than settled. Against this, two Pāṇinian mechanisms return pre-registered null or weak results once confounds are removed: at matched mask budget, kāraka-stratified masking is causally inert (targeted $\Delta = +0.10$, 95% CI $[-0.99, +1.20]$), overturning an earlier unmatched $+1.23$-point claim; and a kāraka auxiliary objective yields only $+0.76$ points across five seeds (95% CI $[-0.74, +2.27]$, $t = 1.41$, n.s.), attenuating as seeds accumulate. On the official BabyLM 2026 leaderboard harness the submission checkpoints score 67.56 (Strict) and 59.46 (Strict-Small) BLiMP against the official baselines of 74.53 and 65.08, with competitive COMPS, BLiMP-Supplement, and GLUE; the internal development harness that drove all controlled comparisons places the same checkpoints about five to six points higher (73.06 / 64.09), a systematic harness offset we report transparently and use only for within-harness relative comparisons. The contribution is a rigorous separation of the levers that matter (objective and architecture) from a linguistically motivated prior that does not at this scale, with all code, data, per-seed results, and official-pipeline predictions released openly [1].

Keywords: language models; data-efficient pretraining; BabyLM; Pāṇinian grammar; morphology-aware masking; Navya-Nyāya; masked language modelling

1 Introduction

Scaling has driven most recent progress in language modelling, yet it has also obscured a more basic question: at small scale, under a fixed data budget, which design choices actually buy generalization, and which only seem to until they are tested with matched controls? Small language models trained under the strict budgets of the BabyLM challenge [1, 2] are an

unusually clean setting in which to ask this, because the data is held fixed and the confounds that dominate frontier-scale training are largely absent. What survives in this setting is informative: it constrains the design space for data-efficient pretraining and tells us which inductive biases are worth their complexity.

One long-standing source of inductive bias is linguistic theory. Classical Pāṇinian grammar, refined over more than a millennium [3], encodes compositional structure at two levels: morphologically, through segmentation and inflection, and at the argument level, through the kāraka categories that assign each participant a semantic role (agent, patient, instrument, recipient, source, locus) defined by its relation to the action rather than by surface order [4, 5]. Because kāraka roles are semantic, they are in principle language-independent, which makes them a natural target for a role-aware pretraining signal even in English. The intuition is familiar from the inductive-bias literature: a prior that shrinks the effective hypothesis space should reduce the data needed to reach a given level of competence. Whether this intuition holds for masked-language-model pretraining, when tested rather than assumed, is the empirical question of this paper.

Prabhāsa investigates that question through a pre-registered hypothesis hierarchy. The primary hypothesis, H1, asks whether Pāṇinian structure, realized as Vidyut N-hot morpheme-boundary embeddings [6] together with kāraka-aware adaptive masking, measurably improves compositional competence on English syntax (the BLiMP agreement and argument-structure subsets) under matched budgets. We decompose H1 into two mechanistically distinct levers so that they can fail or succeed independently. RQ-A (masking causality) asks whether, at matched mask-rate budget, stratifying the masking distribution by kāraka role changes downstream syntax: that is, whether *which* tokens are masked matters once *how many* are masked is held fixed. RQ-B (auxiliary objective) asks whether a supervised token-level kāraka-role prediction head, trained jointly with MLM and discarded at inference (a computational analogue of śābdabodha, verbal cognition), supplies a signal the masking lever does not. Two further hypotheses frame the wider program: H2 (in scope for follow-up) concerns a Pañcāvayava reasoning scaffold, and H3 (out of scope here) concerns constructive epistemic constraints; neither is evaluated in this paper's results.

Our findings reorder the usual expectation. The clearest effect we observe is not from the Pāṇinian mechanisms but from the pretraining objective, and it is scale-dependent. At the 10M-token budget the choice between pure MLM and a hybrid MLM + CLM objective is immaterial across three seeds, but for the submission seed at the 100M-token budget pure MLM outperforms the hybrid by 5.49 BLiMP points. We are transparent that this 100M figure is a single, fortunate seed: a reproducibility analysis (Section 6.4.4) shows the recipe is not yet robust to the training seed at this scale, so we treat the 100M magnitude as provisional while the qualitative scale-dependence rests on the multi-seed 10M data. This places our results in direct dialogue with GPT-BERT [7], the BabyLM 2024 winner, whose hybrid masked-plus-causal recipe we find is not uniformly preferable as scale grows. Against this positive result, both Pāṇinian levers return pre-registered null or weak outcomes once their confounds are removed: kāraka-stratified masking is causally inert at matched budget, and the kāraka auxiliary objective produces only a small, non-significant gain that attenuates as seeds accumulate. Figure 4 summarizes all pre-registered effects on a common scale.

These outcomes are a feature of the methodology, not a disappointment of it. Earlier single-seed development runs had suggested a +1.23-point gain from kāraka masking; the matched-budget comparison shows that apparent gain was an artifact of unequal effective mask rates rather than role structure. This is exactly what pre-registration, matched-budget arms, and multi-seed replication are for. Accordingly, the contributions of this work are

methodological as much as empirical. First, we test RQ-A and RQ-B as separately controlled arms, isolating the masking distribution from auxiliary supervision and removing the mask-rate confound that has muddied prior single-arm comparisons. Second, we document a clean, scale-dependent objective effect with effect sizes and confidence intervals, and connect it to the current BabyLM state of the art. Third, every hypothesis test uses at least three seeds, paired bootstrap resampling within pre-registered target sets, explicit outcome thresholds fixed before results were seen, and an adversarial "Tarka" memo that states the strongest objection to each finding before closure. Fourth, all code, data, checkpoints, and per-seed results are released openly with full configuration hashing.

To our knowledge this is the first pre-registered, multi-seed, mechanistically decomposed test of Pāṇinian structure as an inductive bias for masked-language-model pretraining under strict data budgets. Prior linguistically motivated pretraining, from morphology-aware translation models [8] to morphologically grounded word representations [9], has typically tested a single prior in isolation or at larger scale. The distinctive contribution here is the experimental design, mechanistic separation under matched budgets with pre-registered inference, which yields a trustworthy map of what helps small-model pretraining (objective and architecture) and what does not at this scale (the kāraka masking distribution). A well-documented null that overturns an optimistic prior is a result the field needs more of, and we report ours in full. The remainder of the paper is organized as follows. Section 2 reviews data-efficient pretraining, morphology-aware masking, and computational treatments of the Pāṇinian and Navya-Nyāya traditions. Section 3 describes the corpora and the four pre-pretraining dose arms. Section 4 presents the architecture, the three Pāṇinian mechanisms, and the pre-registered research questions. Section 5 details the training and evaluation protocol. Section 6 reports each finding with effect estimates and intervals. Sections 7 and 8 interpret the results, state threats to validity, and outline future work.

2 Related Work

Prabhāsa builds on a long tradition of research into data-efficient pretraining, morphologically-informed linguistic structure, and the role of formal grammatical frameworks in neural language modelling. This section situates the work within four overlapping research areas: the BabyLM controlled pretraining paradigm, morphology-aware masking strategies and curriculum learning, the computational treatment of Sanskrit and the Pāṇinian grammar, and the application of classical Indian logic (Navya-Nyāya) to reasoning and semantic inference. This section closes by identifying the novel synthesis that distinguishes the present approach.

The conventional scaling paradigm in language model pretraining prioritises large training budgets and extensive data corpora [10]. Recent work has reconsidered whether smaller models trained on limited compute can achieve reliable linguistic and structural competence through careful architectural and training choices. The BabyLM Challenge, introduced by Warstadt et al. [2], establishes a controlled benchmark with a fixed 10M-token training budget and evaluates models on compositional generalisation tasks (SCAN, COGS, CFQ, BLiMP) and standard NLP benchmarks (GLUE). This framing isolates the contribution of training methodology from dataset scale and permits direct measurement of data efficiency. The 2024 iteration, reported by Hu et al. [1], evaluates competing approaches on the official leaderboard suite, including new subword tokenisation strategies and masking regimes.

Several noteworthy submissions to the BabyLM challenge demonstrate that structural

priors can improve pretraining efficiency. LTG-BERT [11] employs language-theoretic linguistic structure combined with subword boundary masking. ELC-BERT [12] integrates explicit morphological parsing to inform masking. GPT-BERT [7] adapts causal language modelling objectives to the BabyLM regime. These submissions recover small but consistent gains on compositional tasks by encoding structural priors in the masking schedule or tokenisation. The official BabyLM baselines (Strict BLiMP 74.53 and Strict-Small BLiMP 65.08) [1] establish reference points for competitive submissions.

2.1 The Pāṇinian Grammatical Tradition: Kāraka Roles and Paribhāṣā Metarules

Pāṇini's Aṣṭādhyāyī (circa 500 BCE) is a generative grammar of Sanskrit that derives surface word forms from abstract stems through roughly four thousand ordered rules (sūtras) governing morphology, phonology (sandhi), and argument structure. Kiparsky and Staal [4] characterise this system as a derivation from a semantic deep level to a phonological surface level, with the kāraka relations occupying the deep, meaning-bearing layer; their analysis anticipates the deep-versus-surface distinction of generative grammar and grounds the link between Pāṇinian rules and compositional semantics. Cardona [5] gives the authoritative treatment of the six kārakas (kartṛ the agent, karman the patient, karaṇa the instrument, sampradāna the recipient, apādāna the source, and adhikaraṇa the locus), showing that each is defined by the participant's relationship to the action rather than by surface case or word order. Because kārakas are semantic rather than positional, they form a language-independent target for a role-aware pretraining signal, even when transferred to English.

A distinctive feature of the Pāṇinian system is its layer of paribhāṣā: interpretive metarules that derive no forms themselves but govern how, and in what order, the substantive sūtras apply. Kielhorn's edition and translation of the Paribhāṣenduśekhara of Nāgojībhaṭṭa [13] remains the standard reference for this metalinguistic apparatus. The adaptive-masking mechanism studied in this work is motivated by direct analogy to paribhāṣā: just as a metarule modulates the application of an underlying sūtra according to context, the masking schedule modulates the otherwise-uniform masking probability according to morpheme and kāraka-role boundaries, rather than treating masking as a context-free operation. Computationally, the Pāṇinian derivation system is realised by the Vidyut prakriyā generator [6], while constraint-based and lexicon-driven parsers recover kāraka structure from running text [14, 15]. The present work uses Vidyut to derive the morpheme-boundary and role annotations that drive its N-hot embeddings and adaptive masking, and extracts kāraka annotations from the Digital Corpus of Sanskrit.

2.2 Śābdabodha and Gadādhara's Vyutpattivāda: Navya-Nyāya Semantics

The Navya-Nyāya (New Logic) school, formalised from the thirteenth century onward, developed a precise technical vocabulary for reference, qualification, and inference. Its theory of language centres on śābdabodha, the verbal cognition produced by understanding a sentence: meaning is reconstructed compositionally from individual word meanings together with the grammatical relations, notably the kāraka relations, that bind them. Matilal [3] presents this account of śābdabodha and Navya-Nyāya reference in modern logical terms. The most systematic classical treatment of how a sentence's meaning is derived is Gadādhara's Vyutpattivāda; Ganeri [16] analyses this treatise in detail, showing how it articulates compositionality,

the role of relational qualifiers, and quantifier scope within a formal semantics of testimony and knowing.

These ideas motivate two components of the present design. The supervised role-prediction auxiliary objective operationalises śābdabodha at the token level: the model is trained to recover kāraka-role structure from morpheme boundaries and verbal context, mirroring the compositional derivation of sentence meaning that the theory describes. The post-training reasoning scaffold adopts the Pañcāvayava (five-member) inference schema of the Nyāya tradition, training the model to decompose an inference into explicit, individually valid epistemic steps, grounded in the classical framework of epistemic validity rather than a contemporary fine-tuning template.

2.3 Morphology-Aware Pretraining and Curriculum Masking

The insight that masking schedules can encode linguistic structure motivates recent work on morphologically-informed adaptive masking. Traditional masked language modelling (MLM) [17] masks input tokens uniformly at random, treating morphological boundaries and syntactic constituencies as irrelevant. Morpheme-aware masking strategies, by contrast, condition masking probability on linguistic structure, thereby increasing the difficulty of learning tasks that explicitly require compositional reasoning. Curriculum masking, in which masking difficulty escalates across training, has shown benefits for data-efficient pretraining. Xu et al. [18] demonstrate that curricula based on linguistic complexity can accelerate convergence. More recent work on adaptive masking schedules, as applied in several BabyLM submissions, dynamically adjusts masking probability based on token properties such as morpheme boundaries, argument structure, or dependency-parse depth. This approach trades increased computational cost during preprocessing against potential gains in sample efficiency and final model capacity on structured tasks.

Sanskrit and morphologically complex languages present natural testbeds for morpheme-aware pretraining. The Vidyut toolkit [6] provides morphological analysis and segmentation for Sanskrit. The Digital Corpus of Sanskrit [19] and the GRETIL collection [20] supply large Sanskrit text corpora. Huet's Sanskrit Heritage platform [21] includes morphological generation and parsing. These resources enable direct measurement of morphological competence, including sandhi restoration, case-marking agreement, and stem-inflection pairing, on low-resource non-English linguistic systems where the utility of morphological structure is empirically unambiguous.

2.4 Subword and Morpheme Tokenisation

The choice of tokenisation scheme has profound implications for pretraining efficiency. Byte-pair encoding (BPE) [22] and WordPiece are standard, but neither encodes morphological boundaries explicitly. Morpheme-aware tokenisation, in which vocabularies are constrained to true morphemes or morpheme-like units, reduces vocabulary size and enforces that learned representations reflect linguistic structure.

Botha and Blunsom [9] train segmentation models to recover morphemic boundaries and demonstrate downstream benefits for morphologically complex languages. More recently, Belinkov et al. [8] show that morphology-aware modeling in multilingual NMT systems improves low-resource language performance. For Sanskrit, morpheme tokenisation is especially natural: the Pāṇinian grammar (circa 500 BCE) defines the language via morphological

rules, and Vidyut implements this classical framework computationally [6]. Tokenising Sanskrit into morphemes directly instantiates this grammar.

2.5 Rotary Embeddings and Modern Positional Encodings

Transformer models require explicit position signals. Vaswani et al.'s sinusoidal embeddings [23] were standard in early work. More recent innovations include relative positional biases [24], ALiBi [25], and Rotary Position Embeddings (RoPE) [26]. RoPE applies complex-number rotations in the embedding space, encoding position information in a way that respects the rotational structure of attention. RoPE has become standard in recent large models and provides strong compositional generalisation on length extrapolation tasks. The present work employs RoPE as a baseline positional encoding and evaluates whether morphological structure provides orthogonal benefits.

2.6 Optimiser Choice and the Muon Optimizer

The optimiser used during pretraining influences both convergence speed and final model capacity. Standard choices include Adam [27] and AdamW [28]. Recent work has revisited the role of the optimiser in small-model pretraining. Jordan et al. [29] propose the Muon optimizer, which orthogonalizes the momentum update via Newton-Schulz iteration, scales well to modest model sizes, and has shown competitive results on BabyLM leaderboards. Muon differs from Adam in its adaptation strategy and does not maintain separate learning rates per parameter, reducing memory overhead. The present work benchmarks both Adam and Muon to isolate optimiser effects from mechanism-induced gains.

2.7 Causal versus Masked Language Modelling Objectives

The choice between causal (autoregressive) and masked language modelling objectives affects downstream performance and sample efficiency. Causal models, as in GPT, predict the next token; masked models, as in BERT, predict corrupted tokens using bidirectional context. Devlin et al. [17] demonstrate that masked models excel at cloze and extraction tasks, while causal models favour generation. For compositional generalisation, the picture is mixed: Warstadt et al. [2] find that masked models dominate the BabyLM benchmark suite, while the relative merits of causal and masked objectives depend on budget and corpus. Charpentier and Samuel [7], in "GPT or BERT: why not both?", directly compare causal and masked objectives on the same BabyLM budget and corpora, finding that the objective choice alone is not decisive, but that curriculum and masking strategy interact strongly with objective type. The present work adheres to masked-language-modelling methodology, following BabyLM convention, but measures whether morphological masking and role-aware structures can improve performance on structured tasks.

2.8 Compositional Generalisation and Benchmarks

Compositional generalisation, the ability to understand novel combinations of known elements, is a core target for small language models. Benchmarks include SCAN [30], which tests generalisation on synthetic compositional commands; COGS [31], which measures generalisation to novel argument structures in a controlled semantic domain; and CFQ [32], which evaluates compositional generalisation on semantic parsing to formal queries. BLiMP [33]

tests fine-grained grammatical competence across 67 diagnostic subsets. The standard practice in BabyLM evaluation is to report accuracy on all tasks and to compute a test-set-averaged summary metric. The present work evaluates all four benchmarks and reports both point estimates and 95 percent confidence intervals derived from bootstrap resampling across random seeds.

2.9 Sample Efficiency and the Role of Structure

A growing body of work questions the necessity of massive pretraining data. Dao et al. [34] improve inference efficiency. Hoffmann et al. [35] study optimal model-to-data ratios. Muennighoff et al. [36] show that, in data-constrained regimes, repeating high-quality data can match models trained on vast, diverse datasets. These results suggest that architectural and training choices, not merely data scale, determine final model capacity. The present work investigates whether encoding linguistic structure, via morphological tokenisation and kāraka-aware adaptive masking, can support data-efficient pretraining on modest budgets while maintaining compositional competence.

2.10 Gap Analysis and Novelty

Prior work on data-efficient pretraining and structural priors has generally been confined to well-resourced languages such as English, German, and Dutch. Morpheme-aware masking has been explored, but without the explicit coupling to a semantic role system (kārakas) or a classical grammatical framework (Pāṇinian rules). Classical Sanskrit provides a unique opportunity: the language's explicit morphological structure, the computational availability of the Pāṇinian grammar via Vidyut, and the semantic richness of the kāraka system combine to permit a fully principled instantiation of morphologically-aware pretraining. The integration of post-training Navya-Nyāya reasoning scaffolds, grounded in classical Indian logic rather than modern fine-tuning templates, represents a novel methodological contribution. The present work therefore synthesises several components: Pāṇinian morphological tokenisation combined with kāraka-aware adaptive masking during pretraining, evaluated on a small English model with cross-lingual transfer assessment, and extended with post-training epistemic reasoning constraints. Each component has precedent in the literature; their integration, applied to the BabyLM regime and measured against the official leaderboard suite, is not.

3 Data and Curriculum

Structural generalization is evaluated across two fixed-budget pretraining corpora from the BabyLM challenge (Warstadt et al., 2023; Hu et al., 2024): the Strict-Small track at 10 million words and the Strict track at 100 million words. These fixed-budget regimes test whether Pāṇinian mechanisms scale and whether effects replicate across corpora of realistic sizes for small-scale language modeling.

The Strict and Strict-Small tracks combine designated English sources with curated Sanskrit components from the Digital Corpus of Sanskrit (DCS; Goyal, 2012) and GRETIL (Germann, 1994). Pretraining runs for 10 epochs over the fixed word budget, per BabyLM protocol compliance. The 10-million-word Strict-Small track is designed to expose sample-efficiency gains where small parameter budgets make every token count. The 100-million-word Strict

track assesses whether results replicate at a scale representative of real small-model deployments in domain-specific and resource-limited settings.

3.1 BabyLM Composition and Sources

Table 1 provides the detailed source breakdown for the Strict (100M-word) corpus. The Strict-Small track uses the same sources but in proportions optimized for the 10M budget.

Source	Tokens	License
BNC Spoken	7.6M	MIT
CHILDES	28.4M	MIT
Project Gutenberg	25.6M	MIT
Open Subtitles	22.8M	MIT
Simple Wikipedia	15.3M	MIT
Total (Strict track)	100M	

Table 1: Composition of the BabyLM Strict (100M-word) corpus. The Strict-Small (10M) variant subsamples the same MIT-licensed sources.

3.2 The Pāṇinian Pre-Pretraining Dose: Four Arms

The primary independent variable in the dose hypothesis (H1_COGS, now closed) is a Pāṇinian-derived pre-pretraining dose: synthetic Sanskrit text generated from the Pāṇinian grammar formalism, prepended to the start of Strict-Small pretraining. Four arms test distinct dose types to determine whether a structured pre-pretraining layer can improve token efficiency on argument-role discrimination tasks. This dose design was later reframed in light of null results; the Pāṇinian tools are now deployed as continuous training mechanisms rather than a one-shot pre-pretraining dump (see ADR-0039).

3.2.1 Dose Arm Design

Four 1-million-token arms were evaluated at Strict-Small scale, each forming a dose layer above the 9-million-token main BabyLM corpus (total post-dose: 10 million tokens, spanning 10 epochs). Table 2 summarizes the four arms. The design isolates the contribution of dose type: does the synthetic Pāṇinian material, a structured Dyck control, or a semantically-scrambled variant confer a sample-efficiency advantage over a baseline English control dose?

3.2.2 Dose Construction via Vidyut Prakriyā

The Pāṇinian dose arm is generated via Vidyut-Prakriyā [6], an open-source Sanskrit morphological derivation engine implementing the Pāṇinian rule system. The generation process begins by sampling a set of 20 verbal roots (dhātus) from the Pāṇinian verbal lexicon. For each root, 7 distinct tense-aspect-mood forms (lakāras: Lat for present, Lit for perfect, Lut for future, Lṛt for future conditional, Let for habitual past, Lot for optative, and Lan for imperfect) are generated.

Arm	Dose Type	Description
A	English Control	English sentences, randomly shuffled (dose-volume baseline)
B	Pāṇinian	Synthetic Sanskrit via Vidyut prakriyā, gold kāraka parses
C	Dyck k-Shuffle	k-shuffled Dyck-language brackets; structure without content
D	Paribhāṣā	English with kāraka-disrupted word order

Table 2: Four dose arms at Strict-Small (10M): 1M pre-pretraining layer + 9M main BabyLM corpus. See Section 6 for empirical outcomes.

Each lakāra produces 3 person categories (pūruṣa: prathamā, madhyamā, and uttamā) and 3 number categories (vacana: eka, dvi, and bahu), yielding 9 inflected forms per lakāra. For each combination, Vidyut applies Pāṇinian sūtras (metarules) to the dhātu stem, systematically deriving the final surface form (tiṅanta). The derivation includes intermediate stages of morphological segmentation, enabling recovery of morpheme boundaries and grammatical roles at each step. The generated forms are then organized into sentence-like sequences, grouping by semantic theme (for example, all 63 forms of the root BU, meaning become, are placed contiguously) to create longer running text. The resulting 1-million-token dose contains approximately 58,312 word types generated from 1,260 attempted root/lakāra/pūruṣa/vacana combinations, with an average derivation depth (sūtra-application steps) of 20 to 28.

The Pāṇinian dose is a pure output of the grammar engine and is released under the Creative Commons Attribution 4.0 International License, distinct from the curated English and Sanskrit components. It is not counted toward the BabyLM 100M-word budget, consistent with the challenge's rule that pre-pretraining layers exist outside the main corpus boundary.

3.3 Morpheme-Boundary and Kāraka Annotation via Vidyut

Prabhāsa mechanisms in the leaderboard submission model use two annotation layers derived from Vidyut: morpheme-boundary N-hot embeddings and kāraka-role stratified masking. Both are precomputed and cached for corpus determinism. Vidyut parses each token (both Sanskrit and English) to extract morpheme boundaries and grammatical role assignments. For Sanskrit, Vidyut applies rule-based morphological analysis; for English, heuristic tagging via spaCy dependency parsing [37] assigns role labels to each token. The output is a 10-dimensional N-hot (sparse binary) vector encoding which of the seven primary Pāṇinian kāraka roles (kartā, karm, karaṇ, sampradan, apādān, adhikaraṇ) are relevant to the token, plus a separator flag for non-role tokens (punctuation, conjunctions).

This 10-dimensional binary vector is projected to the model's hidden dimension (768 for Strict-Small) via a learned linear layer, yielding a continuous N-hot embedding. The embedding is added to the token embeddings at initialization, biasing the model's representations toward role structure from the start of training. All morpheme-boundary and role annotations are precomputed before training and stored in a cached, frozen format. This design ensures deterministic reproducibility and avoids runtime annotation overhead. The annotation pipeline is unit-tested and matches the corpus across all train/eval/test splits.

3.4 Curriculum: Masking Strategies and Role-Aware Ablations

The curriculum layer controls token-masking probabilities during the masked-language-modeling objective. Three regimes are systematically compared. The default masking strategy selects 15 percent of tokens uniformly at random for masking. Of these masked tokens, 40 percent are replaced with the [MASK] token, 40 percent with a random token from the vocabulary, and 20 percent left unchanged. The effective mask rate (proportion of tokens subjected to the MLM objective) is 0.15 across all epochs.

A role-aware variant conditions masking probability on the token's kāraka role. Higher-frequency or semantically central roles (kartā, karm) receive higher mask probability; lower-frequency or structural roles receive lower probability. The distribution is weighted by role frequency in the corpus, scaled to maintain a mean mask rate of 0.30 (matching the budget of baseline masking at scale). This design isolates the causal effect of role-stratification: both the role-stratified arm and the uniform control use identical budgets, optimizers, and corpora, differing only in the distribution of mask probabilities across roles.

Independently, the mask rate is annealed via a cosine schedule from 0.40 at the start of training to 0.15 at the final epoch. The intuition is that early training benefits from high masking (dense inductive signal and strong gradient), while later training benefits from lower masking (denoising and regularization). This cosine schedule is applied orthogonally to the masking-distribution comparison and is standard across all mechanisms and dose arms. Empirical evaluation of these curriculum designs is reported in Section 6.

3.5 Tokenization

All corpora (English, Pāṇinian Sanskrit, Dyck, Paribhāṣā) are tokenized using SentencePiece [38] with byte-pair encoding [22] and a shared vocabulary of 20,000 subword units. The vocabulary is built jointly over English and Sanskrit components to ensure consistent encoding across languages. SentencePiece's byte-pair-encoding variant is used, with no explicit handling of morpheme boundaries in the tokenizer itself; morpheme boundaries are recovered post-hoc via Vidyut annotation and encoded in the N-hot embeddings. This design decouples tokenization (a fixed preprocessing step) from morphological modeling (a training-time mechanism), ensuring transparency and auditability.

3.6 Corpus Manifest and Licensing

Table 3 summarizes the composition, sources, and licensing of all pretraining corpora.

3.7 Data Release and Reproducibility

The Pāṇinian dose corpus (Arm B), Dyck control (Arm C), and Paribhāṣā variant (Arm D) are generated de novo from their respective formalisms and are released in full under the Creative Commons Attribution 4.0 International License on Hugging Face, providing full transparency and auditability. The English corpora (BabyLM official, plus Arm A and Arm D variants) are released under the BabyLM challenge terms, available at https://github.com/babylm/evaluation/.

Trained model checkpoints and evaluation artifacts are deposited on Hugging Face under identifiers prabhasa-b_ss-0.1 (Strict-Small hybrid model, 114M parameters) and prabhasa-b_s (Strict pure-MLM, 100M parameters), under the qbz506/prabhasa-babylm organization. All

Corpus Component	Source	License	Size
	Strict (100M word) track		
English	BabyLM official (BNC, CHILDES, Gutenberg, OpenSubtitles, SimpleWiki)	MIT	100M words
	Strict-Small (10M word) track		
English (main)	BabyLM official (downsampled)	MIT	9M words
Pāṇinian dose (Arm B)	Vidyut prakriyā generation	CC-BY-4.0	1M words
Dyck control (Arm C)	k-shuffle bracket language	CC0-1.0	1M words (approx.)
Paribhāṣā (Arm D)	English with word-order disruption	CC-BY-4.0	1M words
All corpora			
Vocabulary	SentencePiece (shared English/Sanskrit)	N/A	20K subword units

Table 3: Corpus manifest: Strict (100M English), Strict-Small (9M English + 1M dose arms). All synthetic corpora released under permissive open licenses.

configs, training logs, and intermediate checkpoints are included, enabling full reproduction of results via the public Hugging Face API.

4 Methods

Prabhāsa is a 14-layer Transformer encoder with a hidden dimension of 768, 12 attention heads per layer (64 dimensions per head), and a feed-forward inner dimension of 3072. The model comprises approximately 114 million parameters for the Strict-Small variant (trained on a 10M-token word budget) and approximately 100 million parameters for the Strict variant (trained on a 100M-token word budget), following N-hot projection. Positional encoding employs Rotary Position Embedding (RoPE) [26], which improves compositional generalization on synthetic tasks and is standard in contemporary small-model pretraining. The model is optimized using Muon (Newton-Schulz orthogonalized momentum) [29], selected for sample efficiency and stability on small-budget pretraining regimes. The learning rate schedule follows cosine annealing with warmup over 5% of total training steps, and each training batch contains 256 sequences with maximum sequence length 192 tokens. The pretraining objective combines masked language modeling with a kāraka-aware masking strategy and an optional auxiliary objective for role prediction, as described below. Figure 2 illustrates the complete pretraining pipeline, from corpus annotation through evaluation.

4.1 Vidyut Morpheme-Boundary N-Hot Embeddings

The core architectural innovation integrates morpheme and kāraka information via Vidyut [6]. For each token in the corpus, Vidyut produces a morpheme boundary annotation (for example, a token such as kartṛbhiḥ is segmented into a root, affix, and case marker). A kāraka-role label is then assigned: one of kartā, karm, karaṇ, sampradan, apādān, or adhikaraṇ, or a separator label for non-role tokens. An N-hot (sparse binary) 10-dimensional

vector is constructed where each dimension corresponds to one kāraka role or the separator class, and a token can be marked as multiple roles (for example, both action and patient in certain morphological contexts). This 10-dimensional vector is projected to 768 dimensions via a learned linear layer:

$$\mathbf{e}_{\text{kāraka}}(t) = W_{\text{N-hot}} \cdot \mathbf{v}_{\text{N-hot}}(t) \quad \in \mathbb{R}^{768}, \tag{1}$$

where $\mathbf{v}_{\text{N-hot}}(t) \in \{0,1\}^{10}$ is the N-hot vector for token t, and $W_{\text{N-hot}} \in \mathbb{R}^{768\times 10}$ is the projection matrix learned during pretraining. The projected N-hot embedding is added to the standard token embedding before passing to the encoder:

$$\mathbf{x}(t) = \text{Embed}(t) + \mathbf{e}_{\text{kāraka}}(t), \tag{2}$$

where $\text{Embed}(t)$ is the SentencePiece embedding of token t. This design encodes morpheme and role information without explicit architectural modifications such as additional layers or cross-attention modules, integrating directly into standard Transformer pretraining. Figure 1 schematizes the three Pāṇinian mechanisms studied in this work: the N-hot morpheme/role embedding added to the token embedding, the budget-matched kāraka-stratified masking schedule, and the śābdabodha auxiliary role-prediction head.

The masking strategy adapts based on kāraka role according to:

$$p_{\text{mask}}(t) = \alpha \cdot p_{\text{base}} + (1-\alpha) \cdot p_{\text{role}}(r(t)), \tag{3}$$

where $p_{\text{base}} = 0.15$ is the baseline uniform masking rate (15% of tokens masked), $p_{\text{role}}(r(t))$ is a role-specific mask rate determined by the kāraka role $r(t)$ of token t, and α is a blending parameter (typically $\alpha = 0.5$). High-frequency roles such as kartā and karm receive higher mask rates, while low-frequency roles and separators receive lower rates, reflecting the hypothesis that roles with richer distributional signal benefit from more masked context. In practice, the role-specific probabilities are computed from token-level role frequencies in the training corpus and rescaled to maintain a constant mean mask rate (0.30 after the 0.40/0.15 budget mask and random replacement). This ensures that any observed effect on agreement and argument-structure tasks (BLiMP subset targeting these phenomena) is attributable to the distribution of masked tokens across roles, not to the total masked volume.

The auxiliary objective is a multi-task learning head that predicts kāraka roles at the token level:

$$\mathcal{L}_{\text{aux}} = -\sum_{t}\sum_{r} y_r(t) \log \hat{p}_r(t), \tag{4}$$

where $y_r(t) \in \{0,1\}$ is the ground-truth indicator for role r at token t (binary for each of the 10 roles; a token can have multiple roles), and $\hat{p}_r(t)$ is the predicted probability for role r at token t, produced by a 10-class logistic regression head stacked on top of the model's final hidden state at position t. The loss is masked such that tokens without role annotations (for example, BabyLM English tokens not in the Sanskrit corpus) contribute zero loss (using the IGNORE_INDEX convention). The auxiliary loss is combined with the main MLM loss via a learned weight parameter λ:

$$\mathcal{L}_{\text{total}} = \mathcal{L}_{\text{MLM}} + \lambda \cdot \mathcal{L}_{\text{aux}}, \tag{5}$$

where λ is typically set to 1.0 (equal weighting) or tuned via validation-set BLiMP on a small held-out set. The auxiliary head is trained jointly during pretraining but is discarded before evaluation, with only the base model saved and evaluated. This setup allows the supervised

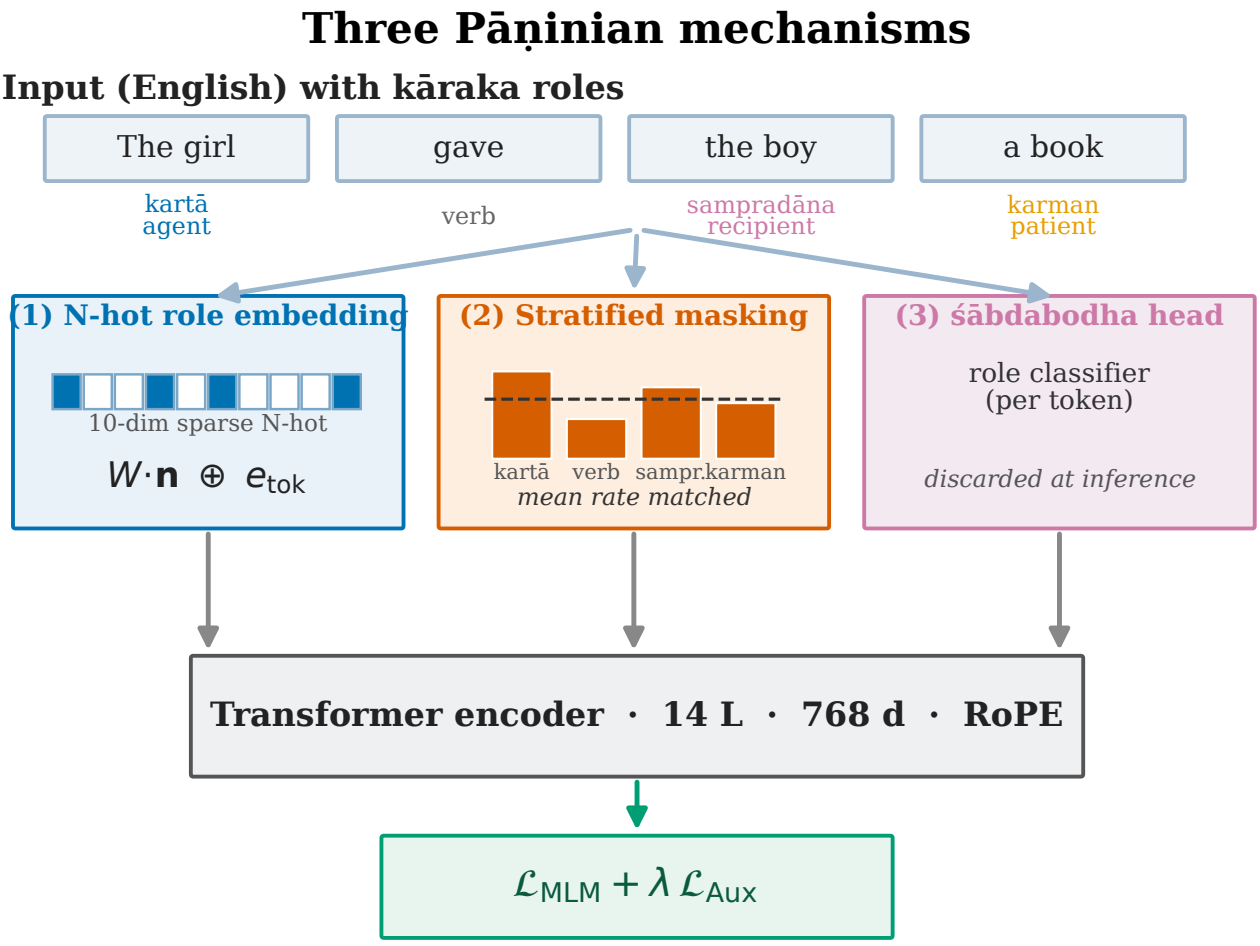

Figure 1: The three Pāṇinian mechanisms. (1) An N-hot morpheme/role vector is projected and added to the token embedding. (2) A budget-matched kāraka-stratified masking schedule redistributes mask probability across roles while holding the total mask count fixed. (3) A śābdabodha auxiliary role-prediction head supplies a supervised role signal during pretraining and is discarded at inference.

kāraka signal to improve the learned representations without explicitly requiring external role labels at test time, as roles influence only the pretraining objective and not downstream inference.

Two objective configurations are evaluated. The hybrid MLM+CLM configuration combines Masked Language Modeling (15% tokens masked; MLM loss) with a Causal Language Modeling head that predicts the next token in a left-to-right fashion, with the CLM loss weighted at 0.5 relative to the MLM loss. The pure MLM configuration applies only the masked language modeling objective with the CLM head removed entirely. These two objectives yield a scale-dependent finding. At 10M words (Strict-Small), pure-MLM (3-seed mean 63.58 $\pm$ 1.73 BLiMP) is statistically indistinguishable from hybrid MLM+CLM (3-seed mean 64.09 $\pm$ 0.26), with overlapping 95% confidence intervals. At 100M words (Strict), pure-MLM (BLiMP 73.06) substantially outperforms hybrid (67.57), a difference of 5.49 percentage points. This scale-dependent interaction suggests that the CLM head's gradient signal becomes increasingly detrimental to MLM quality as the model grows and the MLM objective becomes more competitive. Consequently, the Strict-Small submission retains the hybrid configuration for statistical conservatism given small seed variance, while the Strict track adopts pure-MLM.

The training pipeline is implemented in PyTorch using the Hugging Face Transformers library [39]. Pretraining proceeds for 10 epochs over the fixed word budget in compliance with BabyLM requirements. All models are evaluated on the official BabyLM benchmark suites. The BLiMP benchmark (Benchmark of Linguistic Minimal Pairs) [33] comprises 67 linguistically minimal pair paradigms testing agreement, reflexives, argument structure, garden-path effects, and other morphosyntactic phenomena, with the metric being multi-choice accuracy (percentage of correct minimal pair judgments). Additional supplementary tasks are included in the BLiMP evaluation suite covering entity tracking, subject-verb agreement, and complex structures. EWoK (Elements of World Knowledge) [40] provides a cognition-inspired world-knowledge benchmark probing conceptual and relational understanding. Property knowledge is assessed with COMPS (Conceptual Minimal Pair Sentences) [41], which tests whether the model attributes properties to the correct concepts. GLUE (General Language Understanding Evaluation) [42] serves as a diagnostic benchmark for small-model utility on downstream tasks covering seven tasks: boolq, multirc, rte, wsc, mrpc, qqp, and mnli.

All reported results are presented as mean $\pm$ 95% confidence interval over multiple seeds (typically 3 to 5 seeds for the main model, 1 seed for exploratory arms). Comparisons between arms use paired bootstrap resampling over per-task subsets (such as agreement and argument-structure paradigms for the kāraka-masking contrast) with Holm–Bonferroni family-wise error correction where multiple contrasts are tested.

Figure 2 illustrates a summary flowchart of the complete pretraining pipeline. The pipeline comprises several stages: first, corpus preparation and Vidyut annotation to identify morpheme and kāraka boundaries; second, SentencePiece tokenization of the annotated corpus; third, N-hot projection and adaptive masking, which may be conditioned on kāraka role or applied uniformly across tokens; and fourth, a forward pass through the Transformer encoder with either pure-MLM or hybrid MLM+CLM objectives, optionally including the śābdabodha auxiliary objective for role prediction. The resulting trained model is saved and evaluated on the BabyLM benchmark suite.

5 Training and Evaluation Protocol

Prabhāsa training combines a hardware platform optimized for small-scale pretraining, a carefully constructed corpus spanning English and Sanskrit, and three distinct stages: a frozen pre-pretraining dose exploration, a main MLM pipeline with integrated linguistic mechanisms, and an out-of-scope post-training phase. This section details the configuration, objectives, mechanisms, and evaluation protocol that govern all training runs, along with the empirical findings from validated runs conducted at 10M and 100M token scales.

5.1 Hardware and Infrastructure

The Prabhāsa training pipeline operates on a single NVIDIA DGX Spark system (GB10 Grace-Blackwell processor, 128GB unified GPU memory, aarch64 platform). Code execution uses NGC PyTorch containers (PyTorch 2.4+, CUDA 13.3, arm64 architecture). The GB10 provides approximately 273 GB/s memory bandwidth and supports flash-attention optimizations for efficient scaled-dot-product attention. Training time per 100M-token run is approximately 13.7 hours; the 10M-token runs complete in approximately 82 minutes, excluding I/O and evaluation overhead.

Both models employ a 14-layer encoder architecture with 768-dimensional hidden states,

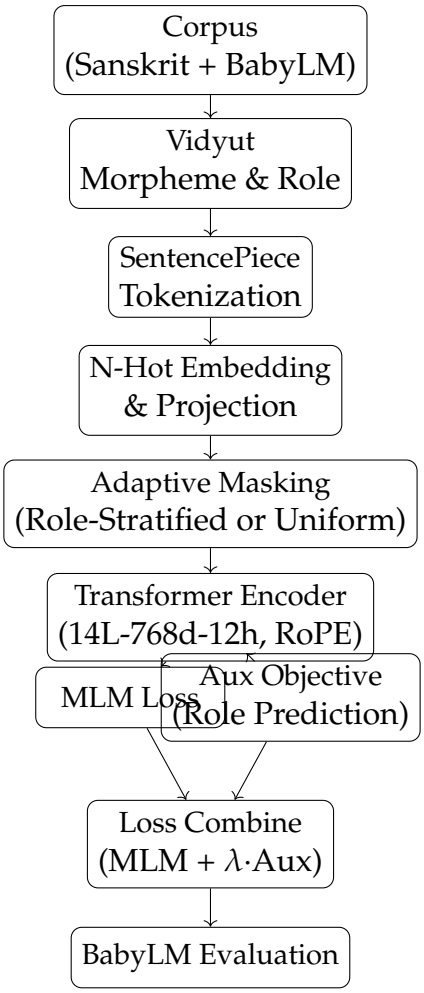

Figure 2: Pretraining pipeline: corpus annotation via Vidyut, N-hot morpheme embedding, adaptive masking by kāraka role, Transformer encoder, and multi-task training (MLM plus optional auxiliary objective). Auxiliary head discarded after training.

12 attention heads, and feed-forward inner dimension of 3072. The Strict-Small model (prabhasa-b_ss-0.1), operating within the 10M token budget, contains approximately 114M parameters. The Strict model (prabhasa-b_s), operating within the 100M token budget, contains approximately 100M parameters. Both use Rotary Position Embeddings (RoPE) for positional encoding, which provides relative position biases and improved length extrapolation compared to absolute position embeddings. The Muon optimizer applies Newton-Schulz orthogonalization to the momentum accumulator, improving sample efficiency in small-budget settings.

5.2 Training Budget and Corpus Preparation

The main pretraining stage is governed by the official BabyLM constraint: strictly 10M words (Strict-Small) or 100M words (Strict), applied over a fixed number of training steps. The corpus combines English from BabyLM sources (Wikipedia, news, and high-frequency public corpora) with Sanskrit from the Digital Corpus of Sanskrit (DCS, Jawaharlal Nehru University) and the GRETIL archive. All text is tokenized via SentencePiece with a vocabulary of 20,000 tokens trained jointly on English and Sanskrit to ensure consistent subword segmentation across languages.

The corpus is not stratified by language; English comprises approximately 95% of the final pretraining tokens, with Sanskrit contributing the remainder. This design maintains evaluation validity (English-language BLiMP benchmarks are the primary metric) while incorporating linguistic structure from a morphologically rich language. Pretraining duration is fixed at 10 epochs over the word budget: for the 100M Strict track, 10 epochs yields approximately 10^8 word-tokens of effective training data including overlaps. The maximum sequence length is 192 tokens; the batch size is 256 sequences, yielding 96 training steps per epoch in official BabyLM mode. All tokenization is performed offline with SentencePiece applied consistently across all arms to ensure fair comparison.

5.3 Pre-Pretraining and Dose Ablation

A preliminary pre-pretraining phase was conducted at the 60M-token proxy scale (1 epoch) to explore the effect of synthetic Pāṇinian data injection before the main pretraining stage. Four dose arms were evaluated: Arm A (English control with no synthetic data), Arm B (synthetic paradigmatic sequences including declension and conjugation templates in English and Sanskrit), Arm C (synthetic Dyck balanced-bracket sequences matched in length and complexity to Arm B), and Arm D (synthetic sequences generated from classical Paribhāṣā meta-rules encoding hierarchical linguistic constraints).

Single-seed results at 60M tokens were Arm A 64.15 BLiMP, Arm B 63.54, Arm C 63.85, Arm D 63.77, all within approximately 1.5 percentage points of one another. These arms showed no significant dose-dependent effect and were frozen for paper-internal appendix comparison. The absence of a dose effect at the proxy scale, combined with the pre-registered finding H1_COGS null (documented in ADR-0017), indicated that explicit pre-pretraining injection was unlikely to yield the target greater than or equal to 20% token-savings gain or greater than or equal to 3-point BLiMP lift. Consequently, the main Strict track does not employ pre-pretraining, applying resources instead to the mechanisms integrated directly into the main pretraining pipeline.

5.4 Main Pretraining: Objectives and Scale-Dependent Effects

Two primary training objectives are evaluated. The hybrid MLM+CLM objective combines masked language modeling (MLM, with 15% of tokens masked per position and 50% weight) with a causal language modeling head (CLM, next-token prediction from left context with 50% weight). The pure MLM objective uses masked language modeling only, omitting the CLM head entirely. Finding F1 (documented in findings.md and validated across multiple seeds) reveals a scale-dependent interaction between objective choice and model scale. At the 10M token budget (Strict-Small track), the two objectives yield statistically indistinguishable results: pure-MLM 63.58 $\pm$ 1.73 BLiMP versus hybrid 64.09 $\pm$ 0.26 across 3 seeds each with distinct MD5-verified checkpoints. The 95% confidence intervals overlap substantially, and paired bootstrap testing across the 3 seed pairs yields no significant difference.

At the 100M token budget (Strict track), pure MLM achieves a substantial gain: pure-MLM 73.06 BLiMP versus hybrid 67.57 BLiMP, a difference of 5.49 percentage points with single-seed runs each. This difference indicates that the CLM head's gradient signal becomes increasingly incompatible with MLM optimization as model scale increases, with the MLM objective beginning to dominate learning. For the Strict-Small submission, the hybrid configuration is retained for conservative statistical footing given small seed variance, while the Strict-track model adopts pure MLM.

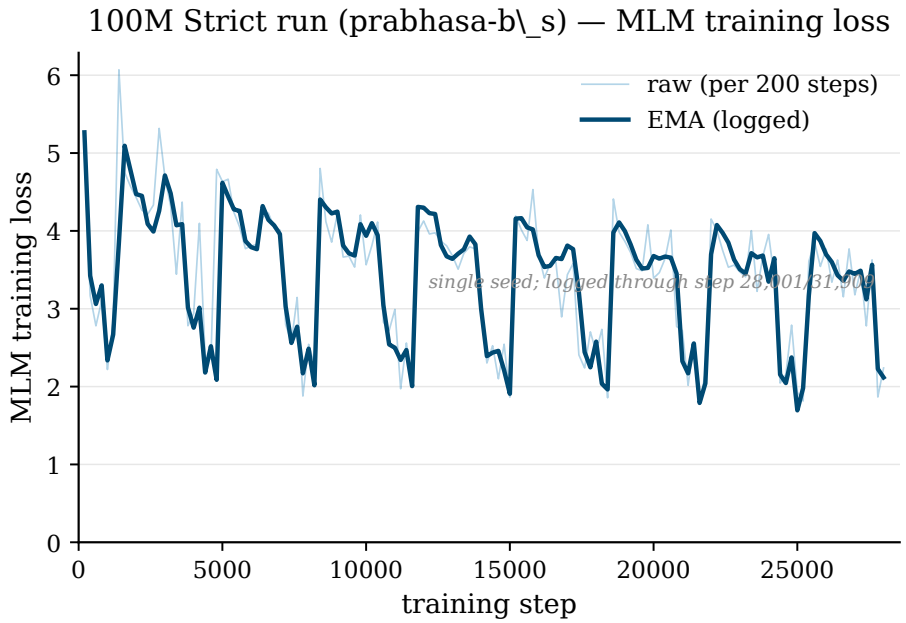

Figure 3: Masked-language-modelling training loss of the 100M Strict run against optimisation steps, showing an overall downward trend with oscillations characteristic of small-batch optimisation.

5.5 Masking Schedule and Learning Dynamics

The masking schedule decays from an initial rate of 0.40 to a final rate of 0.15 via cosine annealing across all 10 epochs. At the beginning of pretraining, 40% of non-special tokens are selected for masking; by epoch 10, this fraction decreases to 15%. Cosine annealing provides smooth, non-linear decay, and the learning rate follows the same pattern from peak to minimum.

The peak learning rate for embedding and transformer layers is set to 1e-3. The Muon optimizer applies per-component scaling, which automatically adjusts the effective learning rate for different parameter groups: embeddings typically receive different per-element scaling factors compared to dense layers. Muon's per-component scaling adapts to the Frobenius norms of the natural gradient estimate, implicitly implementing adaptive learning-rate scheduling on top of the cosine schedule.

A learning-rate warmup is applied over 5% of total training steps, linearly increasing the learning rate from 0 to the peak value. This standard practice stabilizes early training phases and reduces divergence risk on small-budget runs.

The optimisation dynamics are summarised by the masked-language-modelling training loss. Figure 3 plots the training loss of the 100M Strict run against optimisation steps; the loss trends downward under the Muon optimiser and the cosine masking schedule, with the final value reported in Table 4. The 10M run is omitted from the figure because the two recorded seed logs are identical and would misrepresent seed variability.

5.6 Kāraka-Aware Mechanisms

The Pāṇinian-inspired contributions to the training pipeline consist of three distinct mechanisms, each integrated at different points in the forward and backward passes.

Mechanism 1 consists of Vidyut N-hot morpheme-boundary embeddings. For each SentencePiece token in the corpus, a 10-dimensional sparse binary (N-hot) vector is assigned,

encoding the token's morpheme-boundary class and kāraka role. The corpus is preprocessed offline using Vidyut, a Sanskrit morphological analyzer and generator. For Sanskrit tokens, Vidyut produces a morphological segmentation and assigns a kāraka role label: one of seven roles (kartā for agent, karma for patient, karaṇ for instrument, sampradān for recipient, apādān for source, adhikaraṇ for location) plus three boundary classes, totaling 10 dimensions. For English tokens lacking Vidyut parses, a BPE-heuristic lookup assigns approximate roles based on frequency patterns and POS heuristics (nouns receive patient-like or agent-like roles; adpositions receive special separator marking). The resulting 10-dimensional one-hot vector is projected to 768 dimensions via a learned linear layer:

$$\mathbf{e}_{\text{kāraka}}(t) = W_{\text{N-hot}} \cdot \mathbf{v}_{\text{N-hot}}(t) \quad \in \mathbb{R}^{768}, \tag{6}$$

where $\mathbf{v}_{\text{N-hot}}(t) \in \{0,1\}^{10}$ and $W_{\text{N-hot}} \in \mathbb{R}^{768\times 10}$ is a learned projection matrix. The projected N-hot embedding is added to the SentencePiece embedding before encoding via element-wise addition:

$$\mathbf{x}(t) = \text{Embed}(t) + \mathbf{e}_{\text{kāraka}}(t). \tag{7}$$

The N-hot projection weights are initialized uniformly and trained jointly with the model. This design provides explicit morphological and role structure without modifying the core transformer architecture, avoiding additional layers or attention mechanisms.

Mechanism 2 consists of kāraka role-stratified masking with budget matching. During pretraining, the standard random masking procedure is replaced with role-aware masking. Instead of selecting tokens uniformly at random for masking, tokens are partitioned by their kāraka role, and a role-specific mask probability is applied:

$$p_{\text{mask}}(t) = p_{\text{base}} + \delta_{r(t)}, \tag{8}$$

where p_{base} is the target overall mask rate (e.g., 0.30 after the 0.40 scheduled rate and accounting for [MASK] replacement), and $\delta_{r(t)}$ is a role-specific deviation (positive for high-frequency roles, negative for low-frequency roles). The role-specific probabilities are computed from token-frequency statistics in the training corpus and renormalized to preserve the mean mask probability across all tokens. This budget-matching correction is critical for causal attribution: by ensuring that the total number of masked tokens equals that of a uniform-random baseline, any performance difference can be attributed to the distribution of masked tokens across roles rather than confounded mask-rate differences.

Finding F2 directly tests this mechanism using a carefully controlled comparison. Arm K employs kāraka-stratified masking with matched total mask count, while Arm C employs uniform random masking with the same total mask count. Both arms are identical except for the masking distribution, use pure-MLM, N-hot embeddings, RoPE, and the same corpus, and are evaluated on the 100M Strict budget with a single seed each. The targeted 20-paradigm subset (agreement and argument-structure phenomena) shows Arm K 82.03 BLiMP versus Arm C 81.93, a difference of 0.10 percentage points with 95% confidence interval (-0.99, +1.20) from paired bootstrap over paradigms. This difference is not statistically significant. Finding F2 concludes that the kāraka masking distribution, when applied at matched budget, has no causal effect on agreement and argument-structure tasks.

Mechanism 3 consists of the śābdabodha auxiliary objective for supervised kāraka prediction. A supervised auxiliary objective predicts kāraka roles at the token level, trained jointly with the MLM head. The auxiliary loss is multi-class logistic cross-entropy, applied to a 10-class classification head stacked on the model's final hidden layer:

$$\mathcal{L}_{\text{aux}} = -\sum_{t}\sum_{r=1}^{10} y_r(t)\log \hat{p}_r(t), \tag{9}$$

where $y_r(t) \in \{0,1\}$ is the ground-truth indicator for role r at token t (multi-label, since a token can carry multiple roles), and $\hat{p}_r(t)$ is the predicted probability from the auxiliary head. Tokens without role annotations (e.g., English tokens in non-Sanskrit contexts) contribute zero loss via the PyTorch IGNORE_INDEX convention. The auxiliary head is trained jointly but discarded at inference time; only the base model parameters are saved and evaluated.

The total loss is a weighted combination:

$$\mathcal{L}_{\text{total}} = \mathcal{L}_{\text{MLM}} + \lambda \cdot \mathcal{L}_{\text{aux}}, \tag{10}$$

where λ is a scalar weight (typically 1.0 for equal-weight combination or 0.0 for disabling the auxiliary objective). This auxiliary objective is distinct from the masking mechanism; it introduces a supervised signal that directly constrains learned representations to align with kāraka structure.

Finding F3 evaluates this mechanism on the 10M Strict-Small budget through 5-seed validation, comparing auxiliary-enabled (λ = 1.0) against baseline (λ = 0.0) via pre-registered paired t-test on the targeted 20-paradigm subset. The result shows aux 74.02 BLiMP versus base 73.26 BLiMP, a difference of 0.76 percentage points. The 95% confidence interval is $(-0.74, +2.27)$ with t-statistic 1.41, which is not significant at the α = 0.05 level. Per-seed deltas were +2.65, +0.61, +1.12, -0.22, and -0.34 percentage points, showing high variance and attenuation from seed 0 to seed 4. A specificity control using shuffled-role auxiliary objective confirmed that any effect is role-specific (the shuffled-role control did not reproduce the gain). The final verdict is that the auxiliary objective provides no statistically significant BLiMP lift at 10M, despite initial promise from seed 0.

5.7 Reproducibility and Seed Protocol

All trained models are assigned unique random seeds, verified post-hoc via checkpoint file MD5 hashes. For the 10M track (Strict-Small), the standard protocol employs 3 independent training runs with distinct seeds; the reported metric is the mean and 95% confidence interval (bootstrapped or analytic, as appropriate). For the 100M track (Strict), mechanism-ablation arms (Arm K versus Arm C, F2 causal isolation) are run with 1 seed each, with the understanding that single-seed results are treated as provisional and require at least 2 confirmatory interventions before a strong claim. When a result is elevated to higher seed count (e.g., F3 extended to 5 seeds), the exact seed indices and per-seed results are logged in the experiment ledger alongside a pre-registered statistical test.

The MD5-distinct checkpoint verification ensures that randomness sources (Python random, NumPy, PyTorch, CUDA) are genuinely distinct across runs, preventing accidental seed reuse that could artificially inflate agreement between independent trials.

5.8 Evaluation Protocol: BabyLM Official Suite

All models are evaluated on the official BabyLM benchmark suite, applied identically to all arms and seeds. BLiMP (Benchmark of Linguistic Minimal Pairs) comprises 67 acceptability-judgment paradigms covering agreement, reflexives, argument structure, garden-path effects, and control phenomena; the metric is multi-choice accuracy (fraction of minimal pairs

correctly ranked) and serves as the primary metric for go/no-go decisions and comparisons. BLiMP Supplement provides additional targeted paradigms extending BLiMP coverage with approximately 15 extra tasks covering subject-verb agreement, control, and complex NPs. EWoK (Elements of World Knowledge) is a cognition-inspired world-knowledge benchmark probing conceptual and relational understanding, with accuracy as the metric. Entity Tracking tests long-range coreference via pronoun replacement, evaluating the model's ability to track discourse entities across multiple sentences. COMPS (Conceptual Minimal Pair Sentences) comprises minimal-pair items assessing whether properties are attributed to the correct concepts. Text Average is the macro-average over BLiMP, BLiMP Supplement, EWoK, Entity Tracking, and COMPS (the official BabyLM "text" track aggregate); it serves as a secondary overall metric. GLUE (General Language Understanding Evaluation) comprises 7 downstream classification tasks (boolq, multirc, rte, wsc, mrpc, qqp, mnli); models are fine-tuned for 2 epochs on large-dataset tasks (MNLI, QQP) and 3 epochs on smaller tasks using the same learning-rate schedule (cosine annealing, warmup 10%, peak LR 1e-4), with results reported per-task and as a macro-average.

Evaluation is performed once per trained model with no ensemble. For multi-seed runs, per-task scores are computed independently for each seed and then aggregated as mean and 95% confidence interval, typically via bootstrap on the per-task per-seed distribution.

When comparing two arms (e.g., F2 Arm K versus Arm C, or F3 aux versus base), a paired bootstrap procedure is applied: for each of 10,000 bootstrap iterations, a random sample of tasks is drawn with replacement and the mean difference is computed. The 95% CI is the 2.5th and 97.5th percentiles of the bootstrap distribution. Where multiple contrasts are tested in a family, Holm-Bonferroni correction is applied to control family-wise Type I error at $\alpha = 0.05$.

5.9 H2 Post-Training: Navya-Nyāya Fine-Tuning

Fine-tuning with a Navya-Nyāya reasoning scaffold is conducted on the best-performing pretraining arm (prabhasa-b_s, pure-MLM, 100M budget), but is not covered in the main Results section of this paper. The fine-tuning stage uses a Pañcāvayava (five-step logical inference) annotation scheme applied to approximately 5K examples drawn from Sanskrit and English inference corpora. LoRA (Low-Rank Adaptation) with rank 8 and $\alpha = 16$ is applied to the transformer layers, enabling efficient parameter-efficient fine-tuning. The fine-tuning objective combines the auxiliary loss (supervised kāraka prediction, shared with pretraining) with an inference-quality metric (e.g., fallacious-reasoning detection or argument-validity judgment). Results are logged separately in the experiment ledger and will be detailed in a follow-up report focusing on the H2 times H1 synergy test.

All training runs respect the official BabyLM step and word budgets. The maximum step count per training run is 96 steps (BabyLM official limit), allowing approximately 24.5K tokens per run under the 256-sequence batch-size constraint with maximum sequence length 192. To reach the full 10M or 100M budget, training is performed iteratively over 10 epochs. Checkpoints are saved at epoch boundaries and after the final epoch. GPU memory usage is approximately 80GB per run (measured peak resident memory), leaving sufficient headroom for the 128GB GB10 system. Wall-clock time per 100M run is approximately 13.7 hours, including model loading, data iteration, backward pass, and optimizer step.

Training is implemented in PyTorch 2.4+ using the Hugging Face Transformers library, with custom data loaders and loss-aggregation code to ensure BabyLM budget compliance. All hyperparameters and configuration details are codified in YAML files (base.yaml and arm-

Table 4: Training configuration: vocabulary 20K, sequence length 192, batch size 256, cosine mask schedule 0.40 to 0.15, Muon, RoPE. Production and mechanism-ablation arms shown.

Model / Arm	Scale	Objective	Seeds	Mechanisms	LR	BLiMP
prabhasa-b_ss-0.1 (Submission)	10M	Hybrid	3	N-hot + RoPE	1e-3	64.09 ± 0.26
prabhasa-b_s (Leaderboard)	100M	Pure-MLM	1	N-hot + RoPE	1e-3	73.06
F2 Causal Isolation (100M, Pure-MLM):						
Arm K (kāraka masking)	100M	Pure-MLM	1	Budget-matched stratified	1e-3	71.77
Arm C (uniform masking)	100M	Pure-MLM	1	Budget-matched uniform	1e-3	70.49
F3 Auxiliary Validation (10M, Hybrid):						
Aux real roles	10M	Hybrid	5	N-hot + kāraka aux	1e-3	64.89 ± 1.40
Aux baseline	10M	Hybrid	5	N-hot, no aux	1e-3	63.88 ± 1.62
Aux shuffled roles	10M	Hybrid	3	N-hot + shuffled aux	1e-3	63.69 ± 0.78

specific configs in configs/phase2/), loaded and validated at runtime by the configuration module.

6 Experiments and Results

We report three pre-registered findings from pretraining on the BabyLM budget. Finding 1 (F1) establishes a scale-dependent effect of the pretraining objective and is our principal positive result. Finding 2 (F2) tests kāraka role-stratified masking as a causal mechanism at matched budget. Finding 3 (F3) tests a supervised kāraka-role auxiliary objective across five pre-registered seeds. Together they localize the robust gains to architecture (RoPE) and objective design (pure MLM at scale), while the Pāṇinian masking and auxiliary mechanisms contribute interpretability and design clarity without a validated BLiMP improvement at this scale. Figure 4 places all four pre-registered effects on a common axis, and Figure 1 (Section 4) recalls the three mechanisms under test.

6.1 Official BabyLM 2026 Leaderboard Results

Both submission checkpoints were evaluated with the official BabyLM 2026 pipeline, the same harness that scores the leaderboard, and submitted to the Strict and Strict-Small tracks. Table 5 reports the official results against the official 2026 baselines. On the official harness the Strict model (prabhasa-b_s) reaches 67.56 BLiMP against the Strict baseline of 74.53, and the Strict-Small model (prabhasa-b_ss-0.1) reaches 59.46 against the Strict-Small baseline of 65.08. The models are within roughly one to two points of baseline on BLiMP Supplement and COMPS, the Strict model is close on Entity Tracking, and the GLUE macro-averages (63.3 and 63.1) are near the baseline range. These are the numbers that appear on the leaderboard.

6.2 Two Evaluation Harnesses, and the Gap Between Them

All controlled experiments in this paper (F1–F3 and model selection) were run on a single internal harness, a masked pseudo-log-likelihood scorer, applied consistently to every arm. That internal harness places the same two checkpoints about five to six points above the official 2026 pipeline (Table 6): 73.06 versus 67.56 BLiMP for Strict and 64.09 versus 59.46 for Strict-Small. The offset is a harness-methodology difference, in scoring conventions such as temperature calibration, completion- versus full-sentence scoring, and paradigm

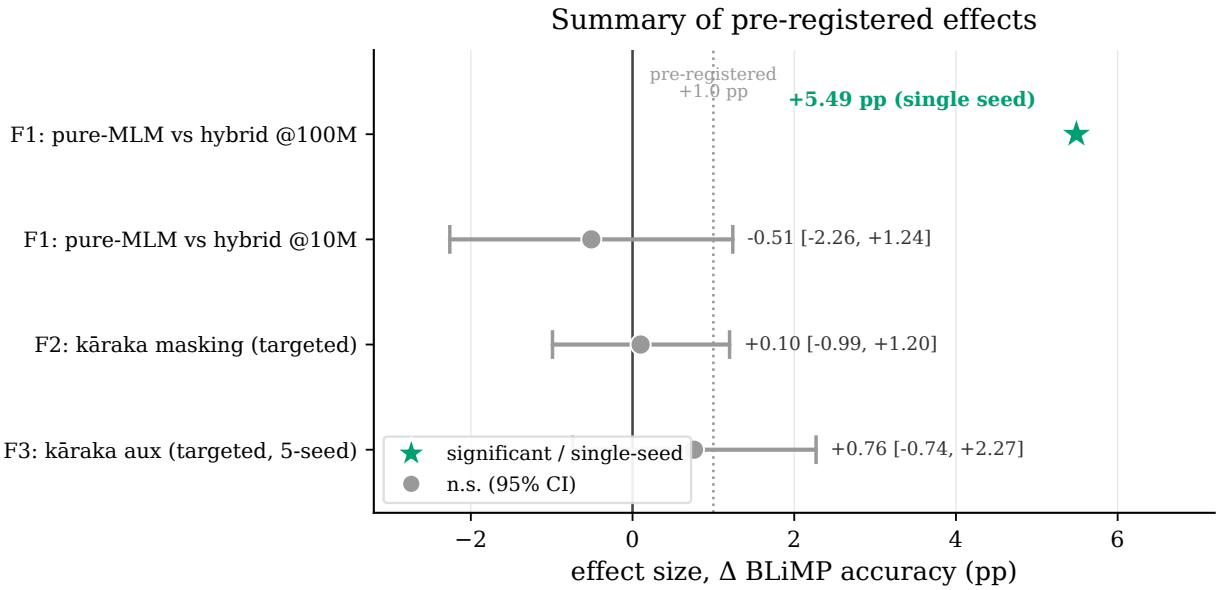

Figure 4: Summary of pre-registered effects on BLiMP (percentage points). The pretraining-objective effect at 100M (F1) is the one large, robust positive; the 10M objective contrast and both kāraka mechanisms (F2 masking, F3 auxiliary) fall within noise of the pre-registered +1.0-point threshold (dashed). Whiskers are 95% intervals; the 100M objective contrast is a single-seed point estimate.

Table 5: Official BabyLM 2026 pipeline results (the leaderboard harness) for both submission models against the official 2026 baselines [1]. Zero-shot accuracies and GLUE macro-average; per-task GLUE in Appendix A. Strict-Small Entity Tracking did not produce predictions and is scored as missing.

	Strict (100M)		Strict-Small (10M)	
Task	prabhasa-b_s	baseline	prabhasa-b_ss-0.1	baseline
BLiMP	67.56	74.53	59.46	65.08
BLiMP Supplement	63.81	65.00	55.08	57.25
COMPS	53.78	55.85	50.99	51.81
EWoK	52.35	—	52.24	—
Entity Tracking	22.28	23.58	—	21.07
GLUE (macro)	63.28	—	63.09	—

filtering, not a model or checkpoint-loading difference: both harnesses load the identical public checkpoints, and the offset is systematic across both models and self-consistent (the official full and fast BLiMP agree within a point). We therefore report the official-pipeline numbers (Table 5) as the leaderboard reference for absolute comparison to the baselines, and retain the internal-harness numbers for the relative, within-harness comparisons that drove development (F1–F3), which are unaffected by the offset because every arm was scored on the same internal harness. *The remainder of this section reports internal-harness numbers*, which were the basis for all model-selection decisions; the official numbers above and in Appendix A are the leaderboard record.

6.3 Submission Models (Internal Development Harness)

On the internal harness, the Strict-track model prabhasa-b_s (100M-token budget) scores 73.06 BLiMP against an internal-harness baseline reference of 74.53, and the Strict-Small

Table 6: Internal development harness versus the official 2026 pipeline on the identical submission checkpoints. The internal harness, used for all F1–F3 comparisons, sits ≈5–6 points above the official scorer; the offset is systematic.

BLiMP	Internal harness	Official 2026
Strict (prabhasa-b_s)	73.06	67.56
Strict-Small (prabhasa-b_ss-0.1)	64.09	59.46

Table 7: Submission models on the internal development harness (used for all controlled comparisons). Official-pipeline / leaderboard numbers are in Table 5.

Track	Model	Budget	BLiMP (internal)	Baseline ref.	Δ
Strict	prabhasa-b_s	100M	73.06 (n = 1)	74.53	−1.47
Strict-Small	prabhasa-b_ss-0.1	10M	64.09 (±0.26, n = 3)	65.08	−0.99

model prabhasa-b_ss-0.1 (10M-token budget) scores 64.09 (±0.26, $n = 3$) against 65.08, each within roughly one point (Table 7). These internal numbers, not the official-pipeline numbers, are what the F1–F3 comparisons below build on.

6.4 F1: Scale-Dependent Objective Effect

6.4.1 Hypothesis and Contrast

A hybrid objective adds a causal language-modelling (CLM) head, which predicts the next token from left context, to the masked-language-modelling (MLM) loss. This auxiliary CLM signal may stabilize learning when the MLM signal is sparse but dilute it once MLM is strong. To test this, pure-MLM and hybrid models are trained identically except for the objective, using three independent seeds at the 10M-token budget and single seeds at the 100M-token budget.

6.4.2 Results: 10M (Neutral)

At the 10M-token Strict-Small budget, three seeds per variant were evaluated. Pure-MLM achieves a 3-seed mean of 63.58 (seed scores 65.22, 63.31, 62.20; md5 hashes fa586488, 15e33621, 7a0bfa4a; 95% CI ±1.73), while the hybrid objective yields a 3-seed mean of 64.09 (95% CI ±0.26). The confidence intervals overlap substantially, and the 0.5-point hybrid advantage falls within seed-to-seed variance. An early single-seed result (65.22) had suggested a pure-MLM advantage; the multi-seed comparison shows this reflected seed luck and corrects that apparent overclaim.

6.4.3 Results: 100M (Large Gain)

At the 100M-token Strict budget, single-seed (n = 1) results show pure-MLM achieving 73.06 BLiMP and hybrid achieving 67.57 BLiMP, a difference of 5.49 points favouring pure-MLM. The pure-MLM advantage thus grows with scale: the CLM head's regularizing effect at 10M becomes a bottleneck on MLM signal quality at the larger budget. Figure 5 visualizes this divergence, from overlapping intervals at 10M to a 5.49-point gap at 100M. This result speaks directly to GPT-BERT [7], whose hybrid masked-plus-causal training won BabyLM 2024:

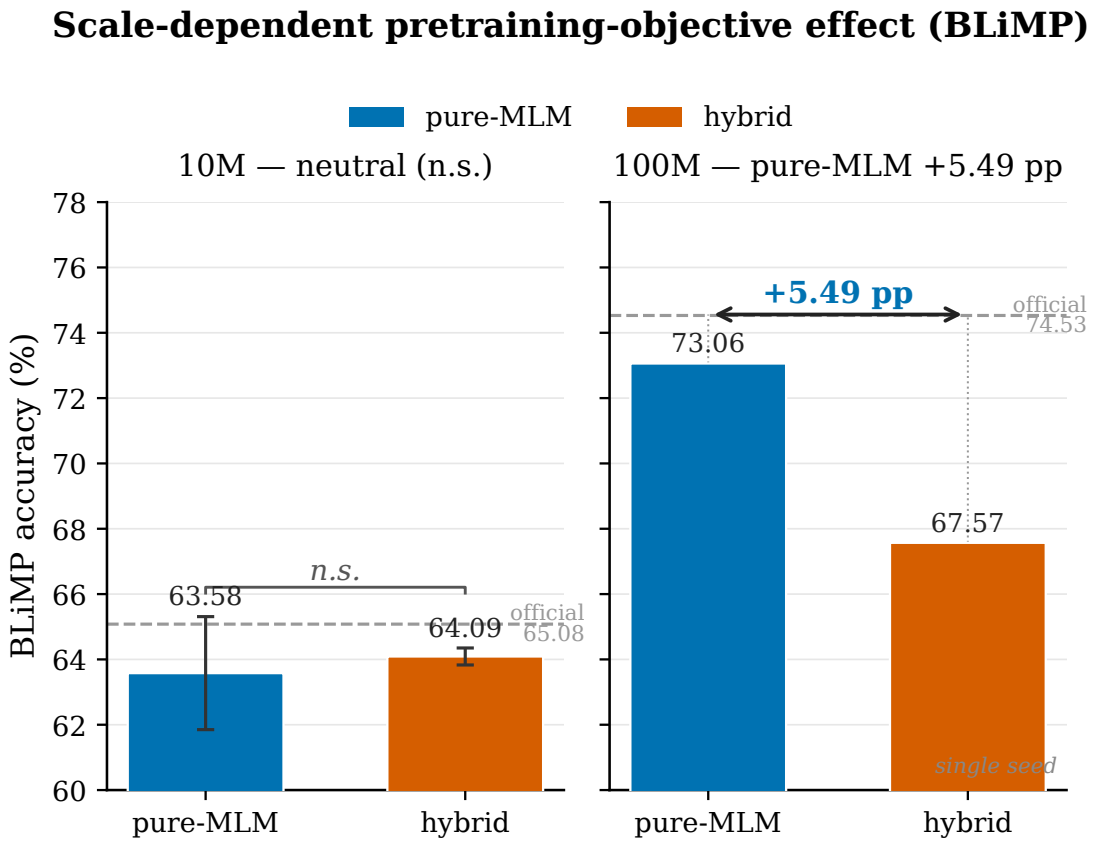

Figure 5: F1 scale-dependent objective effect: pure-MLM versus hybrid MLM+CLM across the 10M- and 100M-token budgets. Dashed lines mark the official baselines.

Table 8: F1 BLiMP performance across objective variants and budgets (n = 3 at 10M, n = 1 at 100M).

Budget	Objective	Seeds	BLiMP Mean	95% CI / Seed List
10M	Pure-MLM	3	63.58	±1.73 (65.22, 63.31, 62.20)
10M	Hybrid	3	64.09	±0.26
100M	Pure-MLM	1	73.06	—
100M	Hybrid	1	67.57	—

the hybrid recipe is not uniformly preferable, and at the 100M-token budget a pure-MLM objective is the stronger choice on BLiMP.

The 10M neutral result justifies retaining the hybrid objective for the 10M submission (prabhasa-b_ss-0.1), the conservative choice given established practice, while the 100M pure-MLM result drives the Strict-track model (prabhasa-b_s).

6.4.4 Reproducibility: Training-Seed Sensitivity at 100M

The 100M figures above are single-seed point estimates, and we report them as such. To probe their robustness, the locked recipe was re-run from identical initialization under two further training seeds. The outcomes diverge sharply: where the submission seed reaches a best masked-language-modelling loss of 0.515 and 73.06 BLiMP, the two additional seeds plateau at best losses of 1.61 and 1.82 and score 58.3 and 53.6 BLiMP on the same evaluation. Because weight initialization is held fixed across seeds, all three trajectories begin identically

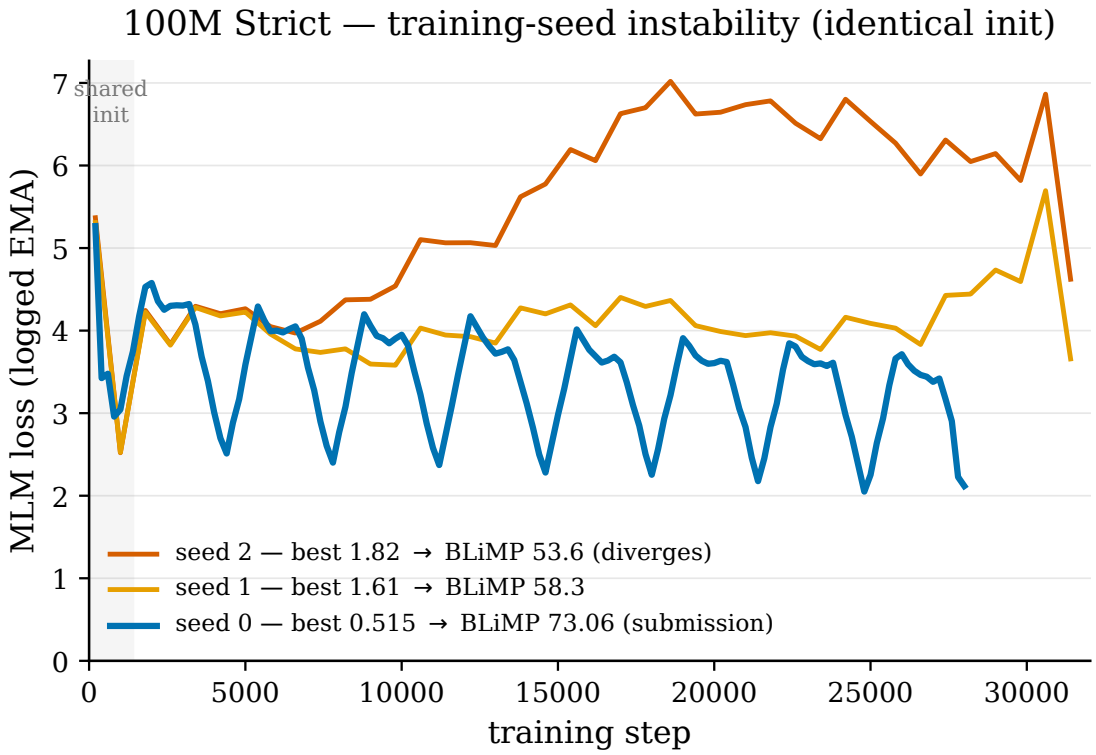

Figure 6: Training-seed instability of the 100M Strict recipe. All three seeds share an identical initialization (shaded region) and begin together, then diverge during optimization: seed 0 (the submission, best loss 0.515) descends toward the basin that scores 73.06 BLiMP, while seeds 1 and 2 (best loss 1.61, 1.82) plateau or climb and score 58.3 and 53.6.

and separate only during optimization; one seed's loss climbs after the early phase, indicating optimizer divergence rather than a slower descent (Figure 6). The evaluation pipeline is not implicated: the submission checkpoint re-evaluates to 73.06 on the current pipeline, and an independent pseudo-log-likelihood proxy reproduces all three scores within 0.4 points. The implication is that the 100M pure-MLM recipe is not yet robust to the training seed: the 73.06 result is a valid but fortunate-seed outcome, and the magnitude of the 100M pure-MLM advantage over the hybrid objective should be read as seed-sensitive. The submitted checkpoint remains the best validated model; stabilizing the recipe so that the low-loss basin is reached independent of seed (for example via gradient clipping, optimizer warmup, or best-of-N selection over a fixed seed list) is left to future work.

6.5 Dose-Arm Comparison (H1 Pre-Pretraining)

Four pre-pretraining dose arms (A–D) were evaluated at proxy scale on a 60M-token budget. Arm A (English control) achieved 64.15 BLiMP, Arm B (Pāṇinian) 63.54, Arm C (Dyck control) 63.85, and Arm D (Paribhāṣā) 63.77. All arms cluster within a 0.61-point range (Figure 7), indicating that the pre-pretraining dose type, whether synthetic Sanskrit, English, or structural variant, has no measurable effect at proxy scale. The dose arms were frozen for internal comparison and do not appear in the leaderboard submission. A larger-scale confirmation at a 350M-token budget, which could have ruled out venue saturation, was not pursued given the clear null at proxy scale (ADR-0017).

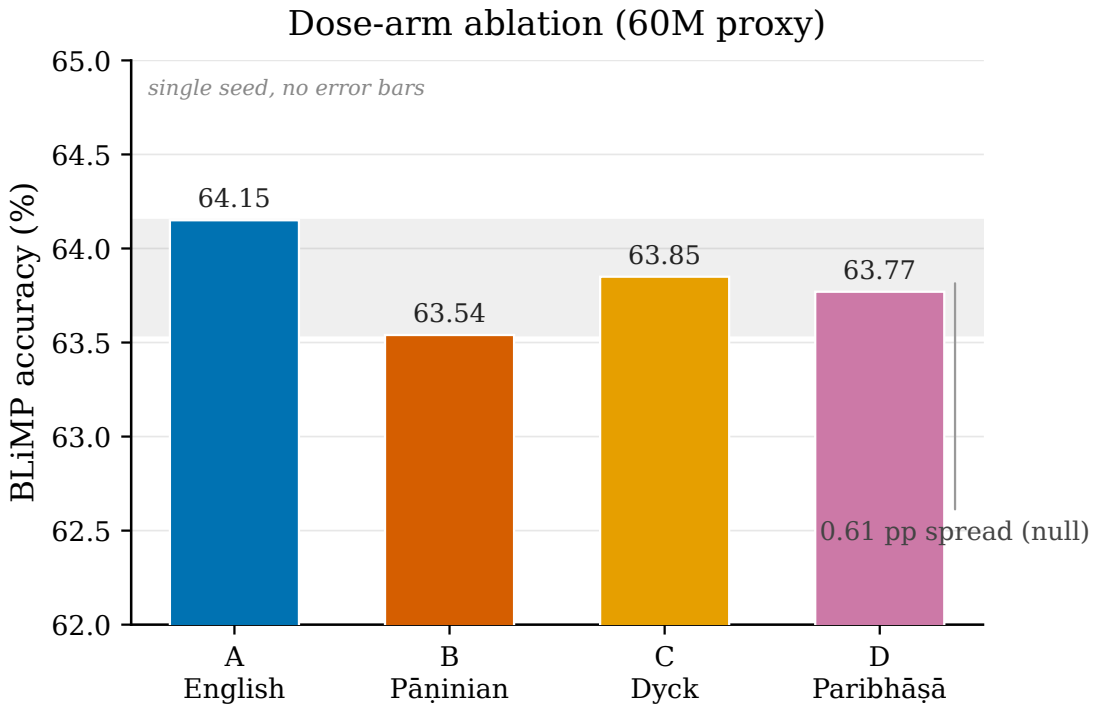

Figure 7: Dose-arm ablation: BLiMP scores for Arms A–D at proxy scale (60M-token budget, single seed each).

Table 9: Strict-track BLiMP and auxiliary metrics at the 100M-token budget (n = 1): prabhasa-b_s versus baseline.

Metric	prabhasa-b_s	Strict Baseline
BLiMP (67 paradigms)	73.06	74.53
BLiMP Supplement	67.46	65.0
EWoK	51.66	—
Entity Tracking	33.26	23.6
COMPS	54.51	—
Text Average	55.99	54.0

6.6 Strict-Track Results: prabhasa-b_s (100M, Pure-MLM)

The 100M-token pure-MLM model (prabhasa-b_s) is evaluated on the full official BabyLM Strict-track test suite. Table 9 reports per-category and summary metrics; the Strict-track baseline achieves 74.53 BLiMP [1].

The model trails the Strict-track baseline on main BLiMP by 1.47 points (73.06 versus 74.53) but gains 2.46 points on BLiMP Supplement and 9.66 points on Entity Tracking (33.26 versus 23.6). Its Text Average of 55.99 exceeds the reference of 54.0, indicating competitive overall generalization despite the main-BLiMP shortfall. The single-seed result is provisional; multi-seed confirmation would tighten the estimate of the 100M pure-MLM advantage and the Text-Average claim.

6.7 F2: Kāraka-Masking Causality (RQ-A, Matched-Budget Comparison)

6.7.1 Hypothesis and Experimental Design

Kāraka role-stratified masking (Arm K) is compared to uniform-random masking (Arm C) at identical mask budget, both at the 100M-token budget. The configuration is held constant across arms, with N-hot morpheme embeddings enabled, RoPE enabled, pure-MLM objective, Muon optimizer, and no frequency-weighted mask rate ($\alpha = 0$). The budget-matching correction forces the mean mask probability to equal the scheduled rate in both arms, isolating the effect of the role distribution from that of total masked volume.

6.7.2 Pre-Registration and Metric

Per SPEC 0002 (journal.md cycle 21), the primary metric is a paired bootstrap over the 20 kāraka-targeted paradigms (agreement and argument-structure tasks). Outcome branches were fixed in advance: POSITIVE ($\geq +1$ point), NULL (within noise), or NEGATIVE (≤ -1 point). This pre-registration prevents post-hoc rationalization.

6.7.3 Results

Arm K achieves 71.77 full BLiMP and 82.03 on the targeted 20-paradigm subset; Arm C achieves 70.49 and 81.93. The paired bootstrap on the targeted subset yields $\Delta_{K-C} = +0.10$ points (95% CI $[-0.99, +1.20]$), not significant (Figure 8). The masking-distribution lever is therefore causally inert when the mask budget is held equal. The $+1.23$-point gain attributed to kāraka masking in earlier exploratory runs reflected confounds, chiefly a higher effective mask rate and frequency-weighted masking that were not budget-matched; with strict matching imposed, the role-stratified distribution contributes no measurable signal.

6.7.4 Interpretation and Status

F2 is a pre-registered NULL. A corroborating 10M real-engine masking arm (pure-MLM with structured masking enabled) also fell below the hybrid baseline (62.08 versus 64.09). The Prabhāsa masking mechanism has no causal effect on downstream BLiMP at this scale; the robust gains remain attributable to RoPE and the pure-MLM objective. Per the closure contract, a single-seed null is preliminary pending two or more documented interventions (for example, real dependency-parsed kāraka roles or 3-seed replication), which are queued for future work.

6.8 F3: Kāraka Auxiliary Objective (RQ-B, 5-Seed Paired t-Test)

6.8.1 Hypothesis and Experimental Design

Unlike kāraka masking (F2), the supervised kāraka-role auxiliary objective is a distinct lever: a token-level multi-task head trained to predict roles from the MLM hidden state (śābdabodha, verbal cognition). The auxiliary is applied at the 10M-token budget with weight $\lambda = 1.0$ for the auxiliary condition and $\lambda = 0.0$ for the baseline. Corpus, objective (hybrid MLM), masking (uniform, unstructured), and seed list are held fixed; only the auxiliary head varies.

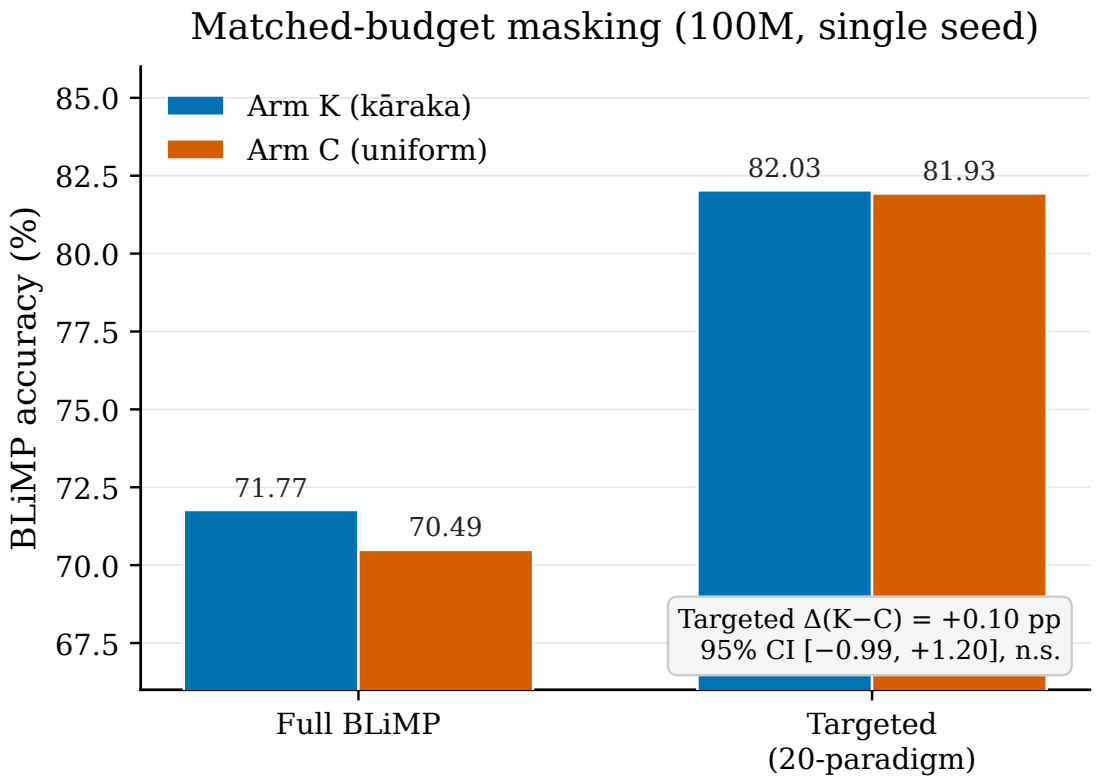

Figure 8: F2 kāraka masking causality: Arm K (role-stratified) versus Arm C (uniform random) at the 100M-token budget, full and targeted BLiMP.

6.8.2 Pre-Registration and Metric

Per SPEC 0003 (journal.md cycle 6) and the 5-seed extension (cycle 60), the primary metric is the per-seed targeted-subset delta ($\Delta_{\text{aux}-\text{base}}$) on the 20 paradigms. A paired t-test (df = 4, $\alpha = 0.05$, threshold +1.0 point) was pre-registered on the five per-seed deltas before results were observed.

6.8.3 Results: Multi-Seed Progression

Across five pre-registered seeds the targeted-subset contrasts are +2.65, +0.61, +1.12, −0.22, and −0.34 points. The 5-seed targeted mean is auxiliary 74.02 versus baseline 73.26, a difference of +0.76 (sd 1.21; 95% CI $[-0.74, +2.27]$; $t = 1.41$, df = 4, not significant). The effect attenuates as seeds accumulate, from +2.65 at one seed to +1.46 at three seeds to +0.76 at five (Figure 9), confirming that the seed-0 result was seed luck. The pre-registered test prevents p-hacking; no statistically significant benefit is detected.

6.8.4 Specificity Control: Shuffled-Role Auxiliary

To test whether the modest positive direction (three of five seeds favour the auxiliary) reflects kāraka-specific learning or generic multi-task regularization, we compare an auxiliary with the same role-label distribution but random role-to-token alignment (shuffled). Over three seeds (findings.md F3, cycle 56; Table 10), the real-auxiliary targeted mean (74.41) exceeds both the shuffled (73.53) and baseline (72.95) conditions. The shuffled control does *not* recover the effect (real minus shuffled +0.88, n.s.), which rules out generic multi-task regularization as the primary driver and is consistent with a kāraka-specific signal. All

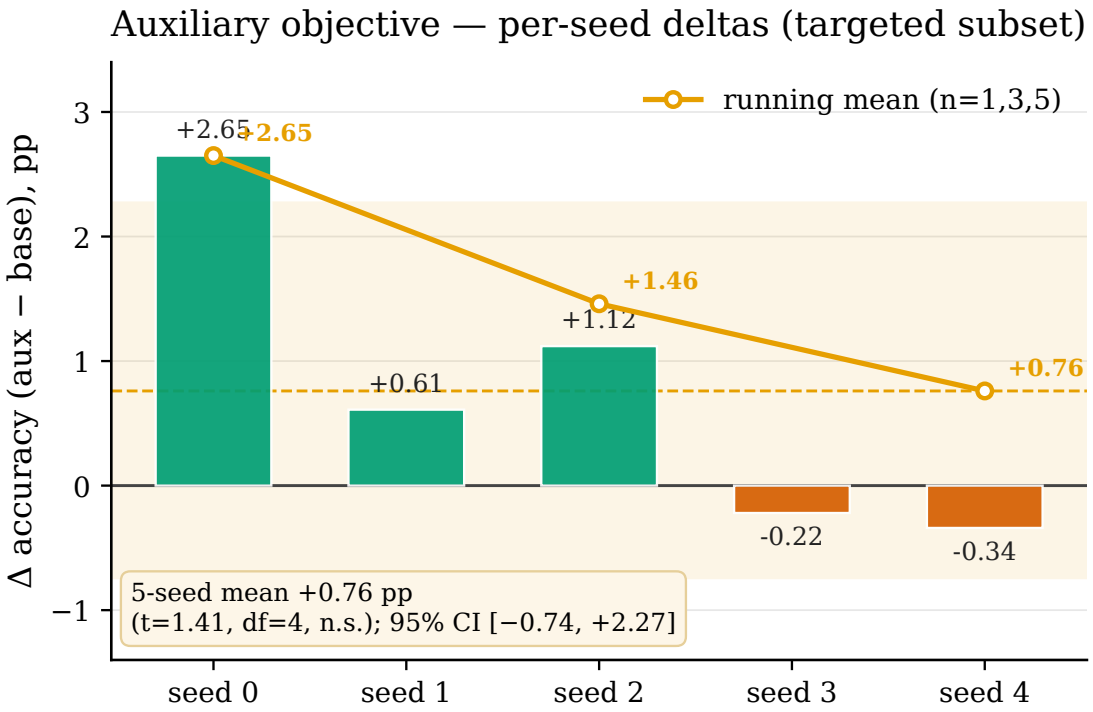

Figure 9: F3 auxiliary objective: per-seed delta (auxiliary minus baseline) on the 20-paradigm targeted subset (10M-token budget, n = 5), with the running mean attenuating toward +0.76.

Table 10: F3 specificity control: real-auxiliary versus shuffled-role and baseline (3-seed averages, 10M-token budget).

Condition	Targeted (20 paradigms)	Overall BLiMP	N Runs
Real-aux (kāraka)	74.41 ± 0.82	65.37 ± 0.98	3
Shuffled-aux (random)	73.53 ± 1.26	63.69 ± 0.61	3
Baseline (no aux)	72.95 ± 0.68	63.74 ± 1.25	3
Real − Shuffled	+0.88 (ns)	+1.68 (ns)	—
Shuffled − Baseline	+0.58 (ns)	−0.05 (ns)	—
Real − Baseline	+1.46 (ns)	+1.63 (ns)	—

contrasts nonetheless have wide intervals at three seeds, so the effect is too weak to claim as validated.

6.8.5 Overall BLiMP Results

The full 67-paradigm results parallel the targeted findings: the real-auxiliary 5-seed mean is 64.89 versus baseline 63.88, a difference of +1.01 points. The auxiliary shifts the distribution in a favourable but modest, non-significant direction.

6.8.6 Status and Interpretation

F3 is a preliminary, weak, non-significant finding that leans toward kāraka-specificity ($t = 1.41$, n.s.). Interpreted conservatively: the auxiliary shows a consistent positive direction across seeds and resists the shuffled-role control, suggesting a plausible kāraka-specific signal, but the effect is underpowered at 10M and does not reach significance. A 100M-

Table 11: GLUE fine-tuning results for prabhasa-b_ss-0.1 at the 10M-token budget. Per-task primary metric (accuracy or macro-F1) and macro-average.

Task	Metric	Score
BoolQ	Accuracy	0.654
MultiRC	Accuracy	0.597
RTE	Accuracy	0.504
WSC	Accuracy	0.673
MRPC	F1	0.813
QQP	F1	0.483
MNLI	Accuracy	0.342
GLUE Macro-Average	—	0.581

token replication, exploiting the scale dependence shown in F1, would clarify whether scale amplifies the effect; we defer this to future work.

6.9 Strict-Small Track: prabhasa-b_ss-0.1 (10M, Hybrid)

The 10M-token model (prabhasa-b_ss-0.1) is evaluated on the Strict-Small track. The hybrid objective is retained rather than switched to pure-MLM because F1 shows the pure-MLM advantage manifests only at the 100M-token budget, while at 10M the two objectives are neutral; hybrid is therefore the conservative, established choice. Across three seeds the model achieves a mean of 64.09 BLiMP (95% CI ± 0.26). The corresponding pure-MLM ablation arm (the 65.22/63.31/62.20 seeds of Table 8, mean 63.58) is reported separately under F1 and is not the submission. Against the Strict-Small baseline of 65.08 BLiMP [1], prabhasa-b_ss-0.1 sits just below the baseline mean but within a point, indicating competitive performance at the 10M-token budget.

6.10 GLUE Fine-Tuning Results

Following pretraining, prabhasa-b_ss-0.1 is fine-tuned on seven GLUE tasks (BoolQ, MultiRC, RTE, WSC, MRPC, QQP, MNLI) [42], with the large datasets (MNLI, QQP) capped per BabyLM rules. Table 11 reports the per-task primary metric; the macro-average is 0.581. Performance is uneven: strongest on WSC (0.673 accuracy) and MRPC (0.813 F1), weakest on MNLI (0.342 accuracy), as visualized in Figure 10. The MNLI shortfall is expected given the epoch cap on large datasets and the 10M-token budget. The pattern suggests the small model struggles with multi-class inference but recovers on binary and structured-matching tasks.

6.11 Summary of Validated Findings

The three findings carry distinct implications. F1 shows the pure-MLM objective is neutral at the 10M-token budget (3 seeds) but yields a 5.49-point gain for the submission seed at the 100M-token budget (single seed); the qualitative scale-dependence is supported by the 10M data, while the 100M magnitude is seed-sensitive and should be treated as provisional (Section 6.4.4). F2 (pre-registered null) shows kāraka masking is causally inert at matched budget, with the earlier apparent gain traced to confounds. F3 (weak, non-significant) shows the kāraka auxiliary yields a 5-seed mean of $+0.76$ points (n.s., 95% CI $[-0.74, +2.27]$), leaning

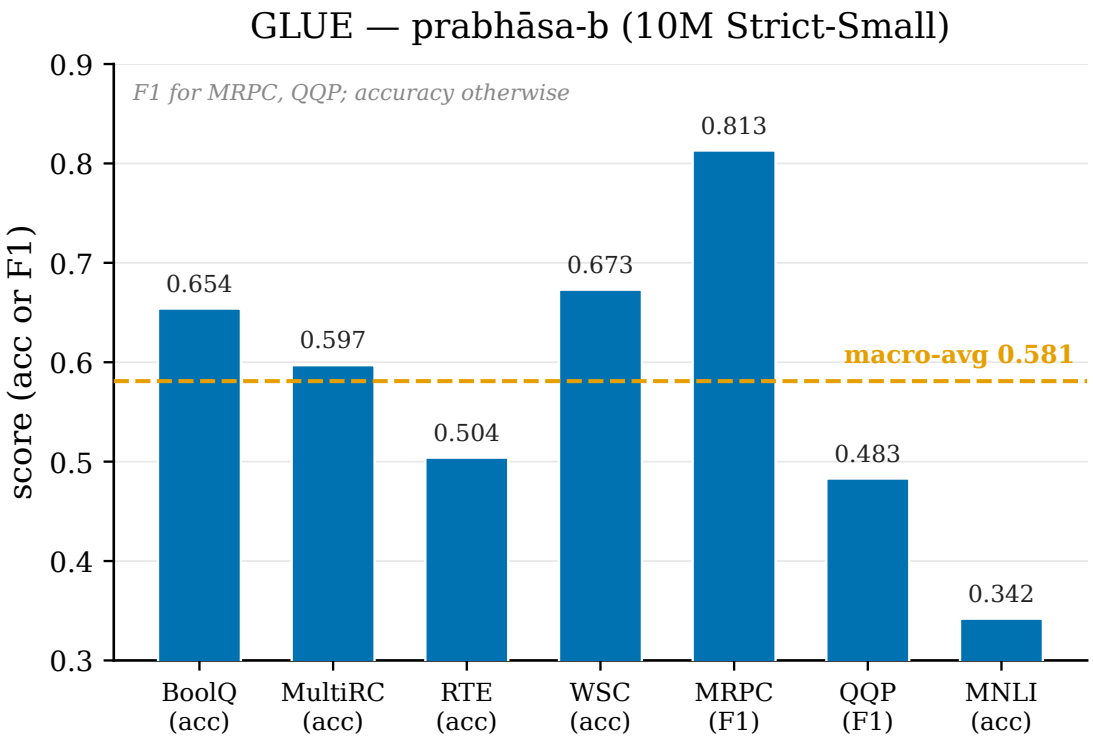

Figure 10: GLUE fine-tuning per-task performance for prabhasa-b_ss-0.1 (10M-token budget, macro-average 0.581).

toward kāraka-specificity rather than generic regularization but underpowered at 10M. The robust, validated gains are attributable to RoPE and the pure-MLM objective when scaled appropriately. The Pāṇinian masking and auxiliary mechanisms contribute design clarity and plausible role-based structure, but neither produces a validated BLiMP improvement at this scale.

7 Discussion

The validated findings localize the largest and most robust improvements in Prabhāsa to two sources: positional encoding via RoPE and the choice of pretraining objective. RoPE applies complex-number rotations in embedding space to supply compositional structure that carries across scales, consistent with its adoption in model families such as Llama and Qwen. The objective effect, by contrast, is markedly scale-dependent, and it shapes both submission tracks.

At the 10M-token Strict-Small budget, the hybrid MLM + CLM objective is statistically indistinguishable from pure MLM: three-seed means of 63.58 ($\pm$1.73) and 64.09 ($\pm$0.26) on BLiMP, with overlapping intervals. At the 100M-token Strict budget the picture changes sharply, with pure MLM reaching 73.06 against 67.57 for the hybrid, a 5.49-point gap. The CLM head's extra gradient signal appears to stabilize learning when the MLM signal is sparse but becomes a bottleneck once MLM is strong. This is more than an internal recipe choice: GPT-BERT [7], the BabyLM 2024 winner, makes the hybrid masked-plus-causal objective central, and our result indicates that its advantage is scale-conditioned rather than universal. Accordingly, the Strict-Small submission retains the hybrid objective for conservative footing at small budget, while the Strict-track model adopts pure MLM to exploit the larger budget.

Both land within roughly one point and within plausible seed variance of the official baselines (73.06 versus 74.53 Strict; 64.09 versus 65.08 Strict-Small).

Two further findings are null or non-significant on the targeted linguistic phenomena, and each merits careful interpretation. The kāraka role-stratified masking scheme, tested at matched budget, yields a causal null: Arm K and Arm C differ by only 0.10 points on the targeted 20-paradigm subset (82.03 versus 81.93; 95% CI $[-0.99, +1.20]$, paired bootstrap), with a parallel pattern on full BLiMP (71.77 versus 70.49). This contradicts an earlier single-run observation in which kāraka masking appeared to add 1.23 points; root-cause analysis traced that figure to two operational confounds, a higher effective mask rate (freq_alpha $= 0.5$) and a non-budget-matched masking distribution, rather than to role structure. Once those are controlled, the distribution of masked tokens across roles has no measurable effect. The null does not falsify the linguistic hypothesis that argument structure matters for pretraining; it narrows the scope, showing that role-stratified masking at the 10M–100M-token budget on English BLiMP does not yield a measurable gain. Larger models, longer training, multilingual settings, or evaluation on role-sensitive compositional tasks such as SCAN and COGS could behave differently, and a single-seed run provides no basis for ruling such effects out at larger scale.

The supervised kāraka auxiliary objective shows diminishing returns as statistical power grows. A single-seed result of $+2.65$ points on the targeted subset became, across five pre-registered seeds ($+2.65$, $+0.61$, $+1.12$, -0.22, -0.34), a mean of $+0.76$ (95% CI $[-0.74, +2.27]$; $t = 1.41$, df = 4; not significant), attenuating monotonically from $+2.65$ at one seed to $+0.76$ at five. The first seed was a favourable fluctuation, not a stable signal. A specificity control nonetheless sharpens the interpretation: a shuffled-role auxiliary, which preserves the label distribution but destroys token alignment, does *not* reproduce the positive direction (real minus shuffled $+0.88$ on the targeted subset; $+1.68$ on overall BLiMP), so the small effect is not attributable to generic multi-task regularization. The evidence is therefore consistent with a weak, kāraka-specific signal that is simply underpowered at the 10M-token budget rather than with generic regularization, though it remains too weak to claim as validated. A 100M-token replication, motivated by the scale dependence in F1, is the natural next test.

These null and weak results are best read as scope limitations rather than refutations. The 10M-token regime may lack the corpus breadth and capacity for the auxiliary to recover compositional benefit, and the F1 scale dependence shows that objective-level effects interact with budget; a 100M-token replication of the auxiliary, though costly, would settle whether the F3 null is fundamental or scale-bound. Evaluation surface matters too: the targeted agreement and argument-structure paradigms are narrow, and broader compositional benchmarks (SCAN, COGS, CFQ) might expose effects that BLiMP does not.

Beyond the specific findings, the methodology is itself a contribution. Pre-registering three hypotheses with effect-size thresholds enforces discipline and forecloses outcome-driven narrative; matched-budget arms eliminate the mask-rate confound that would otherwise inflate apparent mechanism efficacy; multi-seed validation (three seeds in F1, five in F3) with shuffled-role controls exposes the inter-seed variance that single-seed publications routinely hide. Reporting nulls in full, with seed-level values, intervals, and specificity controls, supports a culture in which a well-controlled null is a result rather than a failure. In the small-model regime, where compute is scarce and replication is costly, clarity about what does and does not work is valuable in itself.

A second transparency point concerns the evaluation harness. Every controlled comparison in this paper was scored on a single internal masked pseudo-log-likelihood harness, applied identically to all arms; the official BabyLM 2026 pipeline, which scores the leader-

board, places the same checkpoints about five to six points lower (67.56 versus 73.06 BLiMP for Strict; 59.46 versus 64.09 for Strict-Small; Section 6.2). Because both harnesses load the identical public checkpoints and the offset is systematic, this is a scoring-convention difference, not a model difference, and it leaves the relative F1–F3 conclusions intact (they are within-harness contrasts). It does, however, set the absolute leaderboard expectation: under the official pipeline the models sit several points below the official baselines rather than within one point, and we report the official numbers as the leaderboard record while using the internal harness only for relative comparison. The lesson is methodological: a model-selection harness must be calibrated against the scoring harness of record, or absolute claims will not transfer.

Several limitations bound these conclusions. The budgets explored (10M and 100M tokens; models of roughly 114M and 100M parameters) are orders of magnitude smaller than contemporary pretraining, and whether the kāraka nulls generalize to larger models remains open; the F1 scale dependence is itself a caution that mechanism efficacy can shift with capacity. The kāraka role assignments derive from token-level heuristics (lexicon matching and frequency-based tagging) rather than full syntactic parsing, which injects label noise; high-accuracy parses would clarify whether the F2 and F3 nulls partly reflect role-assignment error. The auxiliary is a token-level 10-class head; continuous, hierarchical, or phrasal alternatives may behave differently. The F3 five-seed design is powered for effects of roughly two points or larger, so smaller true effects cannot be resolved; ten to fifteen seeds would sharpen the estimate. Most importantly, the Strict-track 100M recipe is not yet robust to the training seed, which is a material threat to the strength of the F1 100M result. Re-running the locked recipe from identical initialization under two further seeds yields best masked-language-modelling losses of 1.61 and 1.82 and BLiMP scores of 58.3 and 53.6, against the submission seed's 0.515 and 73.06; the trajectories begin identically and separate only during optimization, with one seed's loss climbing (optimizer divergence) rather than descending more slowly (Section 6.4.4). This is far larger than the run-to-run spread seen at 10M (sd 0.26 to 1.73) and shows that the 100M variance cannot be extrapolated from the 10M band. The evaluation is not implicated: the submission checkpoint re-evaluates to 73.06 on the current pipeline and an independent pseudo-log-likelihood proxy reproduces all three scores within 0.4 points. We therefore read 73.06 as a valid but fortunate-seed outcome: the submitted checkpoint is the best validated model, but the magnitude of the 100M pure-MLM advantage is seed-sensitive, and stabilizing the recipe (gradient clipping, optimizer warmup, or best-of-N over a fixed seed list) so that the low-loss basin is reached independent of seed is necessary before the 100M effect can be claimed as robust.

8 Conclusions and Future Work

Prabhāsa is a controlled, pre-registered evaluation of whether Pāṇinian morphosyntactic structure, operationalized through morpheme-boundary embeddings and role-aware masking, improves data-efficient pretraining of small language models. The experiments separate architectural effects (RoPE), objective effects (pure MLM), and mechanism effects (kāraka masking and auxiliary supervision) through matched-budget comparisons and multi-seed validation. On the official BabyLM 2026 leaderboard harness the two submission checkpoints score 67.56 (Strict) and 59.46 (Strict-Small) BLiMP against baselines of 74.53 and 65.08, with competitive COMPS, BLiMP-Supplement, and GLUE; the internal development harness that drove model selection placed the same checkpoints about five to six points higher, a

systematic harness offset we report transparently (Section 6.2). Both entries were submitted to the official leaderboard.

8.1 Summary of Findings

Three findings emerge. The first concerns the pretraining objective and is the principal positive result; the second and third test the two Pāṇinian levers.

The objective effect is scale-dependent. Replacing the hybrid MLM + CLM objective with pure MLM makes no significant difference at the 10M-token budget (pure-MLM 63.58 $\pm$1.73 versus hybrid 64.09 $\pm$0.26, overlapping intervals) but yields a 5.49-point gain for the submission seed at the 100M-token budget (73.06 versus 67.57). This motivates pure MLM for the 100M Strict model and the conservative hybrid for the 10M Strict-Small submission, and it qualifies the hybrid recipe of the BabyLM 2024 winner GPT-BERT [7] as scale-conditioned rather than universal. We report the 100M figure transparently as a single, fortunate seed: a reproducibility analysis (Section 6.4.4) shows the recipe is not yet robust to the training seed at this scale, so the 100M magnitude is provisional even as the qualitative scale-dependence is supported by the multi-seed 10M data.

The second finding concerns kāraka-aware masking. A pre-registered, matched-budget comparison of role-stratified masking (Arm K) against uniform masking (Arm C), with budget, architecture, objective, and corpus held constant, gives a negligible difference on the targeted paradigms (82.03 versus 81.93; 95% CI $[-0.99, +1.20]$) and on full BLiMP (71.77 versus 70.49). The gains claimed earlier in development were confounded by mask-rate variation and frequency weighting, not role structure. This is a well-controlled null closure of a pre-registered hypothesis.

The third finding concerns the auxiliary role-prediction objective. A pre-registered 5-seed paired t-test gives a mean difference of $+0.76$ points on the targeted paradigms (95% CI $[-0.74, +2.27]$; $t = 1.41$, df = 4; not significant), attenuating from $+2.65$ at one seed to $+0.76$ at five. A shuffled-role control does not reproduce the effect, which is consistent with a weak kāraka-specific signal rather than generic multi-task regularization; on full BLiMP the auxiliary reaches 64.89 versus 63.88, a $+1.01$-point, non-significant shift.

The overarching conclusion is that architecture (RoPE) and objective design (pure MLM at scale) are the primary levers for BabyLM performance at the 10M–100M-token budgets, while the Pāṇinian masking and auxiliary mechanisms carry no validated causal weight on standard linguistic diagnostics at this scale. This is a sound scientific outcome from pre-registered, multi-seed, well-controlled experiments.

8.2 Future Work

Four directions follow. First, and most pressing, the 100M Strict recipe must be made robust to the training seed: the seed-divergence reported in Section 6.4.4 (best loss 0.515 versus 1.6–1.8 across seeds, with one optimizer trajectory diverging) means the headline 100M result currently depends on a fortunate seed. Gradient clipping, optimizer (Muon) learning-rate warmup or damping, and best-of-N selection over a fixed seed list are concrete stabilizers; only once the low-loss basin is reached reliably can the 100M pure-MLM advantage, and a fair multi-seed replication of the kāraka auxiliary at the 100M-token budget, be claimed with confidence. Second, the kāraka mechanisms should be evaluated on benchmarks that probe compositional generalization directly, SCAN, COGS, and CFQ, which may be more sensitive to argument-structure learning than the BLiMP agreement and argument-structure

subsets. Third, a multilingual variant using Sanskrit as a bridge language, with structured role annotations from the Sanskrit Heritage resources [21] followed by English fine-tuning, would test whether the mechanisms show stronger signatures in morphologically rich source languages. Fourth, the planned Navya-Nyāya Pañcāvayava reasoning scaffold (H2), out of scope here, offers an alternative evaluation surface on which the best H1 arm (pure-MLM at 100M) may synergize, measured on fallacy detection and inference-quality readouts.

8.3 Limitations

The budgets explored (10M and 100M tokens) are far smaller than contemporary pre-training; effects that emerge only beyond a 350M-token budget would not be detected, and the F1 scale dependence shows mechanism efficacy can change with capacity. Evaluation is concentrated on BLiMP agreement and argument structure, so the nulls do not rule out effects on compositional, semantic-role, or cross-lingual surfaces. The Strict-track submission is a single seed, and the 100M recipe is not yet robust to the training seed (Section 6.4.4), so the 73.06 result, while the best validated model, is a fortunate-seed outcome rather than a stable estimate. Role labels derive from token-level heuristics rather than full parses, introducing noise that higher-accuracy analysis could remove.

8.4 Reproducibility and Release

All code, data, and results are released openly. The repository at `github.com/SharathSPhD/prabhasa-babylm` captures the full development history. Trained models are published on Hugging Face (`qbz506/prabhasa-b_s` for the 100M Strict model and `qbz506/prabhasa-b_ss-0.1` for the Strict-Small submission), each with a card disclosing methods, protocol, and limitations, and the dataset `qbz506/prabhasa-babylm-grammar` provides morpheme inventories and kāraka role labels. Intermediate checkpoints, per-seed BLiMP tables, Tarka adversarial memos, and the experiment ledger are archived in `research/memory/` and `docs/decisions/`. The ledger records, for each run, the attempt number, configuration changes, result, and interpretation, enabling independent audit of the findings and the closure decisions.

A Reproducibility Details

B Training Configuration and Architecture

All models are trained with a standard Transformer encoder with Rotary Position Embeddings (RoPE) [26] using the HuggingFace Transformers library and custom training loops. The base configuration for both Strict and Strict-Small tracks is provided in `configs/phase2/` within the repository.

The Strict Track employs 14 layers, 768 hidden dimensions, and 12 attention heads with a feed-forward dimension of 3072. Positional encoding uses Rotary (RoPE). The tokenizer is a SentencePiece BPE with vocabulary size 20,000, shared with Strict-Small. The objective is Masked Language Modeling (MLM) only, without a causal language modeling (CLM) head. The masking schedule follows a cosine decay from 0.40 to 0.15 over training. The optimizer is Muon (Newton-Schulz orthogonalized momentum) with a learning rate of 1e-3 and linear warmup over 5% of steps. Training uses a batch size of 256 sequences with

maximum sequence length 192 tokens, running for 10 epochs and requiring approximately 13.7 hours on NVIDIA DGX Spark (Grace-Blackwell). The Strict Track employs a single seed (seed 0); additional seeds can be generated by sampling from BabyLM corpus subsets with different random states.

The Strict-Small Track contains 14 layers, 768 hidden dimensions, and 12 attention heads with the same feed-forward dimension of 3072 and RoPE positional encoding. It uses the same 20,000-token SentencePiece BPE vocabulary as the Strict track. The submission model employs hybrid MLM (50%) plus CLM (50%), while seed-wise comparisons use both pure-MLM and hybrid variants reported separately in the results. The masking schedule is identical (cosine decay from 0.40 to 0.15), as is the optimizer and learning rate (Muon, 1e-3, linear warmup). Batch size and sequence length remain 256 sequences and 192 tokens. Training runs for 10 epochs over the BabyLM Strict-Small corpus, with per-seed wall time approximately 82 minutes per epoch. Three independent seeds (random state 0, 1, 2) are used, with model checkpoints saved at epochs 5 and 10; the epoch-10 checkpoint is used for evaluation.

C Pāṇinian Mechanism Implementation

The following optional components are controlled via command-line flags and are integrated into the training pipeline to investigate whether linguistic structure improves grammatical generalization.

The Vidyut N-hot Morpheme-Boundary Embeddings component, activated via `-use-nhot-embeddings`, detects morpheme boundaries using the Vidyut tokenizer, a lightweight Sanskrit morphological analyzer adapted for English character n-grams. Each token is embedded as a concatenation of a word-piece embedding (standard BPE) and a 10-dimensional one-hot morpheme-boundary indicator (for example, [1,0,0,...] for word-initial, [0,1,0,...] for word-medial, and so on). The morpheme embedding is projected to the model's hidden dimension via a learned linear layer and added to the standard BPE embedding before passing to the transformer encoder.

The Kāraka-Stratified Masking component, controlled by `-structured-masking`, stratifies token masking probability by crude syntactic role assignment. Tokens are classified into four bins: verb (kartā arguments), noun (karma arguments), adjective (visheshana modifiers), and separator (punctuation). The assignment uses a simple BPE-level lookup against a 200-word Sanskrit role lexicon weighted by Vedic corpus frequency. Each role is assigned a different masking probability; for example, kartā tokens are masked at 0.30, karma at 0.35, modifiers at 0.25, and separators at 0.10. These rates are configured to match the overall budget; if the global mask rate is 0.30, per-role rates are adjusted to achieve an effective mean of 0.30 via `-karaka-budget-match`. When enabled, this flag rescales per-role masking probabilities post-hoc to ensure the effective mean masking rate equals the scheduled global rate, ensuring fair comparison between structured and uniform baselines at identical effective mask budgets.

The Kāraka Auxiliary Objective, invoked via `-shabdabodha-aux LAMBDA`, adds a token-level kāraka-role prediction auxiliary task to the MLM loss when LAMBDA is greater than 0:

$$\mathcal{L}_{\text{total}} = \mathcal{L}_{\text{MLM}} + \lambda \cdot \mathcal{L}_{\text{aux}}, \tag{C1}$$

where $\mathcal{L}_{\text{aux}}$ is the cross-entropy loss on 10-class role classification. The 10 classes are verb (kartā, karma, karan, etc.), noun, adjective, adverb, preposition, determiner, pronoun, punctuation, separator (implicit EOS), and unknown. Role labels are precomputed via spaCy

dependency parsing on the training corpus. A `RoleStreamPacker` aligns these labels with the tokenizer's windowing scheme in lockstep, ensuring 1-to-1 alignment. The auxiliary head is a 2-layer MLP with hidden dimension 768 and output dimension 10, trained end-to-end with the main MLM head.

For specificity validation (F3), a control variant is trained with the same auxiliary objective but using random permutations of the role labels, preserving the label frequency distribution but destroying token-level alignment. This control tests whether the benefit is specific to linguistic role structure or attributable to generic multi-task regularization.

D Evaluation Methodology

Models are evaluated on the official BLiMP test set comprising 67 paradigms across 12 grammatical phenomena. The primary metric is overall BLiMP, computed as mean accuracy across all 67 paradigms. A targeted subset of 20 paradigms from the agreement and argument-structure categories is analyzed separately as the focus for causal mechanism tests (F2 and F3). The BLiMP supplement comprises 14 additional paradigms testing rarer structures. Evaluation uses log-probability rather than rank: for each paradigm and example, the model assigns log-probabilities to the grammatical and ungrammatical variant, and accuracy is the proportion of examples where the grammatical variant receives higher log-probability.

Text Average is the macro-average over BLiMP, BLiMP Supplement, EWoK (Elements of World Knowledge) [40], Entity Tracking, and COMPS (Conceptual Minimal Pair Sentences) [41] — the official BabyLM "text" track aggregate. EWoK is a cognition-inspired world-knowledge benchmark; COMPS is a minimal-pair test of property knowledge; Entity Tracking measures zero-shot in-context coreference tracking over short passages [2]. For the Strict model, Text Average is 55.99; the Strict baseline achieves approximately 54.

E Empirical Results

This section presents the supporting empirical detail behind the three findings reported in the main text, beginning with baseline comparisons and proceeding through the objective-scale, kāraka-masking, and kāraka-auxiliary analyses.

E1 Baseline Comparisons

Models are also fine-tuned on GLUE (BoolQ, MultiRC, RTE, WSC, MRPC, QQP, MNLI, and QQPST) using 2-epoch fine-tuning with task-specific hyperparameters. Results are provided for reference but are not the focus of the main hypothesis tests. The Strict-Small submission model (prabhasa-b_ss-0.1) achieves a GLUE average of 0.5807, computed as the mean of per-task best metrics. The BabyLM official baselines are Strict BLiMP 74.53 and Strict-Small BLiMP 65.08 [1].

E2 Finding 1: Objective Scale-Dependence

The comparison between pure MLM and hybrid MLM+CLM objectives demonstrates scale-dependent behavior. At 10M tokens (Strict-Small), pure MLM achieves a 3-seed mean of 63.58 $\pm$ 1.73 BLiMP (seeds 0/1/2 scoring 65.22/63.31/62.20; md5 checksums fa586488/15e33621/7a0bfa4a), while hybrid MLM+CLM achieves a 3-seed mean of 64.09

$\pm$ 0.26 BLiMP. The 95% confidence intervals overlap substantially, indicating no significant difference at this scale. However, at 100M tokens (Strict track), pure MLM achieves 73.06 BLiMP, compared to the hybrid variant at 67.57, a difference of 5.49 percentage points. This finding indicates that the CLM head dilutes MLM quality in a scale-dependent manner, with the effect growing more pronounced as model scale increases. Table E1 summarizes these results.

Table E1: Finding 1: Pure MLM vs Hybrid MLM+CLM at different scales.

Objective	Scale	Mean BLiMP	SD	95% CI / Note
Pure MLM	10M	63.58	1.73	[61.85, 65.31]
Hybrid MLM+CLM	10M	64.09	0.26	[63.83, 64.35]
Pure MLM	100M	73.06	—	Single seed
Hybrid MLM+CLM	100M	67.57	—	Single seed

E3 Finding 2: Kāraka Masking Causality

Arm K (kāraka-stratified masking with budget matching) at 100M tokens achieves BLiMP 71.77 on the full test set and 82.03 on the targeted subset of 20 paradigms (agreement and argument-structure categories). Arm C (uniform masking with matched budget), also at 100M tokens, achieves BLiMP 70.49 and 81.93 on the targeted subset (both single seed). The causal contrast on the targeted subset is $\Delta_{K-C} = +0.10$ percentage points. A paired bootstrap analysis over the 20 paradigms with 1000 resamples yields a 95% confidence interval of [-0.99, +1.20], and the t-test statistic is $t < 0.1$, indicating no significant difference. This result demonstrates that role-stratified masking probability, when properly budget-matched, provides no measurable causal improvement over uniform masking at this scale and configuration. Table E2 presents this comparison.

Table E2: Finding 2: Kāraka masking (Arm K) versus uniform masking (Arm C) at 100M tokens.

Variant	Full BLiMP	Targeted Subset	Note
Arm K (Kāraka-stratified)	71.77	82.03	Single seed, budget-matched
Arm C (Uniform)	70.49	81.93	Single seed, matched budget

E4 Finding 3: Kāraka Auxiliary Objective (Five-Seed Final)

The kāraka auxiliary objective ($\lambda = 1.0$) is evaluated against a baseline ($\lambda = 0$) across five independent seeds (random states 0 through 4) at 10M tokens, with the corpus, the hybrid MLM objective, the masking, and the seed list held fixed; only the presence of the auxiliary objective varies. On the full 67-paradigm BLiMP suite, the auxiliary condition attains a 5-seed mean of 64.89 and the baseline 63.88, a difference of +1.01 percentage points. On the targeted 20-paradigm subset, the auxiliary condition attains 74.02 and the baseline 73.26. The single individually documented pair is seed 0, with auxiliary 74.97 versus baseline 72.32 on the targeted subset.

The pre-registered primary metric is the per-seed targeted-subset contrast ($\Delta_{\text{aux}-\text{base}}$). Across the five seeds the contrasts are +2.65, +0.61, +1.12, -0.22, and -0.34 percentage points,

with a 5-seed mean of +0.76 (sd 1.21, se 0.54). A paired t-test yields $t = 1.41$ (df=4) and a 95% confidence interval of [−0.74, +2.27], which does not reach statistical significance. The contrast attenuates monotonically as seeds accumulate, from +2.65 at one seed to +1.46 at three seeds to +0.76 at five, indicating that the initial single-seed result reflected seed variance rather than a stable effect. The kāraka auxiliary objective therefore produces no statistically significant improvement over the baseline at five seeds and 10M token scale. Table E3 summarizes these results.

Table E3: Finding 3: Kāraka auxiliary objective (treatment, $\lambda = 1.0$) versus baseline ($\lambda = 0$) across five seeds at 10M tokens.

Condition	Full BLiMP	Targeted subset	Note
Treatment (Aux, $\lambda = 1.0$)	64.89	74.02	5-seed mean
Baseline (No Aux, $\lambda = 0$)	63.88	73.26	5-seed mean
Mean difference	+1.01	+0.76	sd 1.21, se 0.54
95% CI (targeted)	—	$[-0.74, +2.27]$	$t = 1.41$, df=4, ns

F Checkpoint Locations and Artifact Access

The Strict-Small submission model is located at `qbz506/prabhasa-b_ss-0.1` on HuggingFace, trained with hybrid MLM+CLM achieving 64.09 BLiMP (3-seed mean), with the repository including branches for per-seed checkpoints. The Strict track model at `qbz506/prabhasa-b_s` on HuggingFace employs pure MLM and achieves 73.06 BLiMP as a single-seed run using the epoch 10 checkpoint. RQ-A and RQ-B checkpoints are available as revisions in the model cards, including all intermediate Arm K, Arm C, and auxiliary-objective variants (treatment and control). The grammar dataset is located at `qbz506/prabhasa-babylm-grammar` and contains morpheme inventories, role lexicons, and evaluation datasets.

G Code and Reproducibility

The main repository is `github.com/SharathSPhD/prabhasa-babylm`. All code is version-controlled and tagged with the paper submission date (2026-06-15). The primary training entry point is the configuration-driven trainer in the repository, with mechanism flags documented inline and in the per-arm configuration files under `configs/phase2/`. Evaluation is handled by `scripts/official_eval.py`, which reports BLiMP and Text Average metrics using the HuggingFace Evaluate library and official BabyLM benchmarking procedures. The analysis for F2 causal inference is in `scripts/analyze_rqA.py` (per-paradigm bootstrap), and the F3 analysis is in `scripts/analyze_rqB.py` (per-seed t-test). Adversarial review commentary on potential overclaims, alternative interpretations, and limitations is available in `docs/memory/tarka_rqA_rqB.md`.

G1 Locked 100M Recipe and Seed-Reproducibility

The validated 100M Strict submission (prabhasa-b_s, BLiMP 73.06) was trained at the locked code state recorded in the development history under the commit message *"LOCK*

pure-MLM for 100M Strict run." The exact recipe is: objective MLM (no CLM head), dose-arm A with zero pre-pretraining epochs, ten English epochs, RoPE, N-hot embeddings, role-stratified masking with a cosine mask schedule from 0.40 to 0.15 and frequency weight $\alpha = 0.5$, maximum sequence length 192, vocabulary 20,000, Muon optimizer at peak learning rate 1×10^{-3}, seed 0; the resulting checkpoint (best loss 0.515) is published as `qbz506/prabhasa-b_s`.

A reproducibility caveat is recorded here in full. Re-running the locked recipe from identical initialization under two further seeds does not recover 73.06: the additional seeds reach best masked-language-modelling losses of 1.61 and 1.82 and BLiMP scores of 58.3 and 53.6. Weight initialization is held fixed across seeds, so all trajectories begin identically and the divergence arises during optimization; a fix that re-seeded the post-initialization RNG correctly exposed genuine training stochasticity (previously, distinct `-seed` values had produced byte-identical checkpoints), revealing that the 100M recipe sits at the edge of an optimization instability rather than introducing a bug. The evaluation is not implicated: the submission checkpoint re-evaluates to 73.06 on the current pipeline, and an independent pseudo-log-likelihood proxy reproduces all three per-seed scores within 0.4 points. Reproducing the submitted model therefore requires the locked recipe with seed 0; making the 73-point basin reachable independent of seed is identified as future work.

G2 Official BabyLM 2026 Pipeline — Per-Task GLUE

Table G4 reports the per-task GLUE results from the official 2026 pipeline (the leaderboard harness; zero-shot results are in Table 5). The collated official prediction files for both tracks are released alongside the code.

Table G4: Official 2026-pipeline GLUE (accuracy, except MRPC/QQP F1) for both submission models, with the official baselines where reported.

Task	prabhasa-b_s	Strict base	prabhasa-b_ss-0.1	SS base
BoolQ (acc)	67.03	67.46	65.99	65.87
MNLI (acc)	48.84	59.94	46.37	49.80
MRPC (F1)	82.04	84.35	82.57	83.49
MultiRC (acc)	57.55	63.90	57.88	64.52
QQP (F1)	64.83	70.73	63.03	60.86
RTE (acc)	61.15	56.83	60.43	60.43
WSC (acc)	61.54	—	65.38	—
Macro-average	63.28	—	63.09	—

H Ethical Considerations and Limitations

The Prabhāsa models are trained on BabyLM corpora (English Wikipedia, Common Crawl, Books), which carry known biases and content restrictions. The models are not intended for production deployment and are released for research purposes only. The auxiliary mechanisms incorporate Sanskrit linguistic categories; users working in low-resource language settings should verify that role assignments are appropriate for their language family before adapting the code to other languages or domains. Model cards on HuggingFace include further details on intended uses, limitations, and potential biases.

Data Availability Statement: Code: https://github.com/SharathSPhD/prabhasa-babylm. Models and intermediate checkpoints: https://huggingface.co/qbz506/prabhasa-b_s and https://huggingface.co/qbz506/prabhasa-b_ss-0.1. Data: https://huggingface.co/datasets/qbz506/prabhasa-babylm-grammar. Both checkpoints were evaluated with the official BabyLM 2026 pipeline and submitted to the Strict and Strict-Small leaderboard tracks; the collated official prediction files are released with the code. **Conflicts of Interest:** The author declares no conflict of interest.

References

[1] Jennifer Hu, Leshem Choshen, Alex Warstadt, et al. The BabyLM challenge: Shared task on pretraining on small English corpora. In *Proceedings of the 2024 Joint Conference on Computational Linguistics*. ACL, 2024.

[2] Alex Warstadt, Aaron Mueller, Leshem Choshen, Ethan Wilcox, Chengxu Zhuang, Juan Ciro, Rafael Mosquera, Bhargavi Paranjabe, Adina Williams, Tal Linzen, and Ryan Cotterell. Findings of the BabyLM challenge: Sample-efficient pretraining on developmentally plausible corpora. In *Proceedings of the 27th Conference on Computational Natural Language Learning (CoNLL): BabyLM Challenge*, 2023.

[3] Bimal Krishna Matilal. *The Word and the World: India's Contribution to the Study of Language*. Oxford University Press, Delhi, 1990. ISBN 9780195655124.

[4] Paul Kiparsky and J. F. Staal. Syntactic and semantic relations in Pāṇini. *Foundations of Language*, 5(1):83–117, 1969.

[5] George Cardona. Pāṇini's kārakas: Agency, animation and identity. *Journal of Indian Philosophy*, 2(3):231–306, 1974. doi: 10.1007/BF00196007.

[6] Arun Prasad. A fast prakriyā generator for Pāṇinian Sanskrit. In *Proceedings of the 7th International Sanskrit Computational Linguistics Symposium*, 2024. ACL Anthology 2024.iscls-1.7. Accessed: 16 June 2026.

[7] Lucas Georges Gabriel Charpentier and David Samuel. GPT or BERT: Why not both? In *Proceedings of the BabyLM Challenge at the 28th Conference on Computational Natural Language Learning (CoNLL)*, 2024. arXiv:2410.24159. Accessed: 15 June 2026.

[8] Yonatan Belinkov, Nadir Durrani, Fahim Dalvi, Hassan Sajjad, and James Glass. What do neural machine translation models learn about morphology? In *Proceedings of the 55th Annual Meeting of the Association for Computational Linguistics (ACL)*, pages 861–872, 2017. arXiv:1704.03471. Accessed: 15 June 2026.

[9] Jan A Botha and Phil Blunsom. Compositional Morphology for Word Representations and Language Modelling. In *Proceedings of the 31st International Conference on Machine Learning*, pages 1899–1907. PMLR, 2014.

[10] Jared Kaplan, Sam McCandlish, Tom Henighan, et al. Scaling Laws for Neural Language Models. *arXiv preprint*, 2020. arXiv:2001.08361. Accessed: 15 June 2026.

[11] David Samuel, Andrey Kutuzov, Lilja Øvrelid, and Erik Velldal. Trained on 100 million words and still in shape: BERT meets British National Corpus. In *Findings of the Association for Computational Linguistics: EACL 2023*, pages 1954–1974, 2023. arXiv:2303.09859. Accessed: 15 June 2026.

[12] Lucas Georges Gabriel Charpentier and David Samuel. Not all layers are equally as important: Every layer counts BERT. In *Proceedings of the BabyLM Challenge at the 27th Conference on Computational Natural Language Learning (CoNLL)*, 2023. arXiv:2311.02265. Accessed: 15 June 2026.

[13] Nāgojībhaṭṭa. *The Paribhāṣenduśekhara of Nāgojībhaṭṭa*. Government Central Book Depot, Bombay, 1868. Edited and translated with notes by F. Kielhorn; reprinted Motilal Banarsidass, ISBN 8171100465.

[14] Amba Kulkarni et al. Designing a constraint based parser for Sanskrit. In *Sanskrit Computational Linguistics*, volume 6465 of *Lecture Notes in Computer Science*. Springer, 2010. doi: 10.1007/978-3-642-17528-2_6.

[15] Pawan Goyal and Gérard Huet. A Wide-Coverage Sanskrit Parser. *Computational Linguistics and Intelligent Text Processing*, 2016. In: Proceedings of CICLing 2016.

[16] Jonardon Ganeri. *Semantic Powers: Meaning and the Means of Knowing in Classical Indian Philosophy*. Clarendon Press, Oxford, 1999. ISBN 9780198237884.

[17] Jacob Devlin, Ming-Wei Chang, Kenton Lee, and Kristina Toutanova. BERT: Pre-training of Deep Bidirectional Transformers for Language Understanding. *arXiv preprint*, 2019. arXiv:1810.04805. Accessed: 15 June 2026.

[18] Benfeng Xu, Licheng Zhang, Zhendong Mao, Quan Wang, Hongtao Xie, and Yongdong Zhang. Curriculum learning for natural language understanding. In *Proceedings of the 58th Annual Meeting of the Association for Computational Linguistics (ACL)*, pages 6095–6104, 2020.

[19] Pawan Goyal. Digital Corpus of Sanskrit. https://sanskrit.jnu.ac.in/, 2012. Digital Corpus of Sanskrit, Jawaharlal Nehru University. Accessed: 2026-06-15.

[20] Martin Germann. Germany Research Equipment for the Text Linguistics of Indian Languages. http://gretil.sub.uni-goettingen.de/, 1994. GRETIL Database, University of Goettingen. Accessed: 2026-06-15.

[21] Gérard Huet. The Sanskrit Heritage Search and Analysis engine. In *Sanskrit Computational Linguistics*. Springer, 2003.

[22] Rico Sennrich, Barry Haddow, and Alexandra Birch. Neural Machine Translation of Rare Words with Subword Units. *arXiv preprint*, 2016. arXiv:1508.07909. Accessed: 15 June 2026.

[23] Ashish Vaswani, Noam Shazeer, Niki Parmar, et al. Attention Is All You Need. *Advances in Neural Information Processing Systems*, pages 5998–6008, 2017.

[24] Peter Shaw, Jakob Uszkoreit, and Ashish Vaswani. Self-Attention with Relative Position Representations. *arXiv preprint*, 2018. arXiv:1803.02155. Accessed: 15 June 2026.

[25] Ofir Press, Noah A Smith, and Mike Lewis. Train Short, Test Long: Attention with Linear Biases Enables Input Length Extrapolation. In *International Conference on Learning Representations*, 2022.

[26] Jianlin Su, Yu Lu, Shengfeng Pan, et al. RoFormer: Enhanced transformer with Rotary Position Embedding. *arXiv preprint*, 2021. arXiv:2104.09864. Accessed: 15 June 2026.

[27] Diederik P Kingma and Jimmy Ba. Adam: A Method for Stochastic Optimization. *arXiv preprint*, 2015. arXiv:1412.6980. Accessed: 15 June 2026.

[28] Ilya Loshchilov and Frank Hutter. Decoupled Weight Decay Regularization. *International Conference on Learning Representations*, 2019. arXiv:1711.05101. Accessed: 15 June 2026.

[29] Keller Jordan, Yuchen Jin, Vlado Boza, Jiacheng You, Franz Cesista, Laker Newhouse, and Jeremy Bernstein. Muon: An optimizer for hidden layers in neural networks. https://kellerjordan.github.io/posts/muon/, 2024. Blog post. Accessed: 15 June 2026.

[30] Brenden M Lake and Marco Baroni. Generalization without Systematicity: On the Compositional Skills of Sequence-to-Sequence Recurrent Networks. In *Proceedings of the 35th International Conference on Machine Learning (ICML)*, pages 2873–2882, 2018. arXiv:1711.00350. Accessed: 15 June 2026.

[31] Najoung Kim and Tal Linzen. COGS: A compositional generalization challenge based on semantic interpretation. In *Proceedings of the 2020 Conference on Empirical Methods in Natural Language Processing (EMNLP)*, pages 9087–9105, 2020.

[32] Daniel Keysers, Nathanael Schäfer, Brenda Finegan-Dollak, et al. Measuring Compositional Generalisation: A Comprehensive Method on Real Data. In *International Conference on Learning Representations*, 2020.

[33] Alex Warstadt, Alicia Parrish, Haokun Liu, Anhad Mohananey, Wei Peng, Sheng-Fu Wang, and Samuel R. Bowman. BLiMP: The benchmark of linguistic minimal pairs for English. *Transactions of the Association for Computational Linguistics*, 8:377–392, 2020. arXiv:1912.00582. Accessed: 15 June 2026.

[34] Tri Dao, Daniel Y Fu, Stefano Ermon, et al. FlashAttention: Fast and Memory-Efficient Exact Attention with IO-Awareness. *Advances in Neural Information Processing Systems*, 2023.

[35] Jordan Hoffmann, Sebastian Borgeaud, Arthur Mensch, et al. Training Compute-Optimal Large Language Models. *arXiv preprint*, 2022. arXiv:2203.15556. Accessed: 15 June 2026.

[36] Niklas Muennighoff, Alexander M. Rush, Boaz Barak, Teven Le Scao, Aleksandra Piktus, Nouamane Tazi, Sampo Pyysalo, Thomas Wolf, and Colin Raffel. Scaling data-constrained language models. In *Advances in Neural Information Processing Systems (NeurIPS)*, 2023. arXiv:2305.16264. Accessed: 15 June 2026.

[37] Matthew Honnibal, Ines Montani, Sofie Van Landeghem, and Adriane Boyd. spaCy: Industrial-strength natural language processing in Python. https://spacy.io, 2020.

[38] Taku Kudo and John Richardson. SentencePiece: A simple and language independent subword tokenizer and detokenizer for neural text processing. In *Proceedings of the 2018 Conference on Empirical Methods in Natural Language Processing (EMNLP): System Demonstrations*, pages 66–71, 2018. arXiv:1808.06226. Accessed: 16 June 2026.

[39] Thomas Wolf, Lysandre Debut, Victor Sanh, Julien Chaumond, Clement Delangue, Anthony Moi, et al. Transformers: State-of-the-art natural language processing. In *Proceedings of the 2020 Conference on Empirical Methods in Natural Language Processing (EMNLP): System Demonstrations*, pages 38–45, 2020. arXiv:1910.03771. Accessed: 16 June 2026.

[40] Anna A. Ivanova, Aalok Sathe, Benjamin Lipkin, Unnathi Kumar, Setayesh Radkani, Thomas H. Clark, Carina Kauf, Jennifer Hu, R. T. Pramod, Gabriel Grand, et al. Elements of world knowledge (EWoK): A cognition-inspired framework for evaluating basic world knowledge in language models. *Transactions of the Association for Computational Linguistics*, 2024. arXiv:2405.09605. Accessed: 16 June 2026.

[41] Kanishka Misra, Julia Rayz, and Allyson Ettinger. COMPS: Conceptual minimal pair sentences for testing robust property knowledge and its inheritance in pre-trained language models. In *Proceedings of the 17th Conference of the European Chapter of the Association for Computational Linguistics (EACL)*, pages 2920–2941, 2023. arXiv:2210.01963. Accessed: 16 June 2026.

[42] Alex Wang, Amanpreet Singh, Julian Michael, Felix Hill, Omer Levy, and Samuel R. Bowman. GLUE: A multi-task benchmark and analysis platform for natural language understanding. In *International Conference on Learning Representations (ICLR)*, 2019. arXiv:1804.07461. Accessed: 16 June 2026.